

# CodroidCS SDK Manual

Version: 2.1.2 | Namespace: `Codroid`

## Table of Contents

- [1. Quick Start](#)
- [2. Core Concepts](#)
- [3. CodroidClient API Reference](#)
- [4. Motion API Reference](#)
- [5. Data Types and Enums](#)
- [6. CRI Real-Time Data and Control API Reference](#)
- [7. IO and Register API Reference](#)
- [8. Utilities API Reference](#)
- [9. .NET Framework 4.6.2 Notes](#)

## Environment Requirements

| Target              | Platform              | Notes                                          |
|---------------------|-----------------------|------------------------------------------------|
| <code>net6.0</code> | Linux, Windows, macOS | .NET 6 SDK                                     |
| <code>net8.0</code> | Linux, Windows, macOS | .NET 8 SDK (recommended)                       |
| <code>net462</code> | <b>Windows only</b>   | .NET Framework 4.6.2+, WinForms/WPF compatible |

## Install via NuGet

```
dotnet add package Codroidsdk
```

## Project Reference

```
dotnet add path/to/YourApp.csproj reference path/to/CodroidSDK/CodroidCS.csproj
```

## API Naming Convention

All public methods return `Task` / `Task<T>` but do **not** use the `Async` suffix.

```
// Correct
await robot.ConnectRemoteAndSwitchOn();
int di = await robot.GetDi(0);

// Wrong
await robot.ConnectRemoteAndSwitchOnAsync(); // does not exist
```

This keeps the same public API names across C#, Python, and C++ SDKs.

---

## Unit Convention

---

| Layer                                 | Linear | Angular       |
|---------------------------------------|--------|---------------|
| SDK public API                        | mm     | deg (degrees) |
| TCP JSON protocol                     | mm     | deg           |
| CRI UDP binary (wire)                 | m      | rad (radians) |
| <code>CriRealTimeData</code> (parsed) | mm     | deg           |

`CriRealtimePacketParser.Parse()` and `CriRealtimeDispatcher` (with `convertToSi=true`) handle the m-to-mm and rad-to-deg conversion automatically.

# 1. Quick Start

---

## Install

---

### NuGet

```
dotnet add package Codroidsdk
```

### Project Reference

```
dotnet add path/to/YourApp.csproj reference path/to/CodroidSDK/CodroidCS.csproj
```

### Supported Targets

```
<!-- net8.0 (recommended) -->
<TargetFramework>net8.0</TargetFramework>

<!-- net6.0 -->
<TargetFramework>net6.0</TargetFramework>

<!-- .NET Framework 4.6.2+ (Windows only) -->
<TargetFramework>net462</TargetFramework>
```

---

## Minimal Example

---

Connect to the controller, read a digital input, write a digital output, and disconnect.

```
using Codroid;

var robot = new CodroidClient("192.168.8.136");

try
{
    // Connect, enter remote mode, and power on
    await robot.ConnectRemoteAndSwitchOn();

    // Read DI port 0
    int di0 = await robot.GetDi(0);
    Console.WriteLine($"DI 0 = {di0}");

    // Write DI value to DO port 10
    await robot.SetDo(10, di0);
}
finally
{
    // Always disconnect in finally
    robot.Disconnect();
}
```

```
}
```

---

## Complete Workflow Example

---

```
using Codroid;

ConsoleUtf8.InitConsoleUtf8(); // Windows console UTF-8

var robot = new CodroidClient("192.168.8.136");

try
{
    // 1. Connect
    await robot.ConnectRemoteAndSwitchOn();

    // 2. IO
    int di0 = await robot.GetDi(0);
    await robot.SetDo(10, di0);

    // 3. Register
    RegisterReadValue reg = await robot.GetRegisterValue(49100);
    int value = reg.GetInt32();
    await robot.SetRegisterValue(49100, value + 1);

    // 4. Motion
    await robot.MovJ(JointPoint.Degrees(new[] { 0, 0, 90, 0, 90, 0 }), speed: 40, acc: 100);

    // 5. Blocking motion
    robot.MovLSync(
        CartesianPoint.MmDegWithRef(new[] { 400, 0, 300, 180, 0, 0 },
robot.CriData.JointPosition),
        speed: 150, acc: 500);
}
finally
{
    robot.Disconnect();
}
```

---

# Run Example Projects

```
# net8.0 (full suite)
dotnet run --project CodroidTestNet8/CodroidTestNet8.csproj

# With controller IP
dotnet run --project CodroidTestNet8/CodroidTestNet8.csproj -- 192.168.8.10

# Specific demo
dotnet run --project CodroidTestNet8/CodroidTestNet8.csproj -- cri 192.168.8.10
dotnet run --project CodroidTestNet8/CodroidTestNet8.csproj -- io 192.168.8.10
dotnet run --project CodroidTestNet8/CodroidTestNet8.csproj -- register 192.168.8.10

# net462 / .NET Framework 4.6.2
dotnet run --project CodroidTestNet462/CodroidTestNet462.csproj -- 192.168.8.10
```

# Error Handling

All TCP commands throw on failure:

| Exception               | Condition                               |
|-------------------------|-----------------------------------------|
| CodroidCommandException | Controller returns err                  |
| TimeoutException        | No response within 10 seconds           |
| ArgumentException       | Invalid parameter (SDK-side validation) |

```
try
{
    await robot.SetDo(999, 1); // Invalid port
}
catch (CodroidCommandException ex)
{
    Console.WriteLine($"Controller error: {ex.ControllerError}");
}
catch (TimeoutException)
{
    Console.WriteLine("Request timed out");
}
```

## 2. Core Concepts

### CodroidClient Lifecycle

```
new CodroidClient(ip)
  |
  v
Connect()  --or--  ConnectRemoteAndSwitchOn()
  |
  v
[ IO / Register / Motion / CRI ... ]
  |
  v
Disconnect()
```

```
var robot = new CodroidClient("192.168.8.136");

try
{
    await robot.ConnectRemoteAndSwitchOn();
    // ... use robot ...
}
finally
{
    robot.Disconnect(); // Always call in finally
}
```

### Constructor

```
var robot = new CodroidClient(string ip);
```

- `ip` -- Controller IP address
- TCP port is fixed at **9001**

### Properties

| Property             | Type                         | Description                                                       |
|----------------------|------------------------------|-------------------------------------------------------------------|
| <code>CriData</code> | <code>CriRealTimeData</code> | Thread-safe clone of CRI data snapshot                            |
| <code>Data</code>    | <code>CriRealTimeData</code> | Direct reference to internal CRI buffer (faster, not thread-safe) |

## Event

```
robot.CriDataReceived += data =>
{
    Console.WriteLine($"Joints: {string.Join(", ", data.JointPosition)}");
};
```

Fires after each valid CRI UDP frame is parsed. The `data` parameter is a thread-safe clone.

## TCP Command Model

Every SDK method that talks to the controller follows this pattern:

1. SDK assigns a unique `id`
2. SDK serializes `{ id, ty, db }` as JSON and sends over TCP
3. Controller responds with `{ id, ty, db, err }`
4. SDK matches the response by `id`
5. If `err` is non-empty, throw `CodroidCommandException`
6. If no response in 10s, throw `TimeoutException`

## CommonResponse

```
public class CommonResponse
{
    public object? id { get; set; } // Request ID
    public string? ty { get; set; } // Response type
    public JsonElement db { get; set; } // Business data
    public string? err { get; set; } // Error message
}
```

Most methods return `Task<CommonResponse>`. The `db` field contains the actual result data.

## Unit Convention

SDK public APIs use **mm** and **degrees**. This matches the TCP JSON protocol.

| Context                               | Linear | Angular |
|---------------------------------------|--------|---------|
| SDK API, TCP JSON                     | mm     | deg     |
| CRI UDP wire format                   | m      | rad     |
| <code>CriRealTimeData</code> (parsed) | mm     | deg     |

**Important:** CRI UDP binary payloads use meters and radians. The SDK automatically converts to mm/deg in `CriRealtimePacketParser.Parse()` and `CriRealtimeDispatcher` (with `convertToSi=true`). Do not assume raw UDP floats are in mm/deg.

## Async Naming Convention

All public methods return `Task` or `Task<T>` but do **not** use the `Async` suffix.

```
// These are async methods -- await them
await robot.ConnectRemoteAndSwitchOn();
int di = await robot.GetDi(0);
await robot.MovJ(JointPoint.Degrees(joints), 40, 100);
```

This design keeps the same API names across C#, Python, and C++ SDKs.

## Exception Types

| Exception                                | When                                      | Source         |
|------------------------------------------|-------------------------------------------|----------------|
| <code>CodroidCommandException</code>     | Controller returns <code>err</code> field | TCP response   |
| <code>TimeoutException</code>            | No response within 10 seconds             | TCP wait       |
| <code>ArgumentException</code>           | Invalid parameter value                   | SDK validation |
| <code>ArgumentOutOfRangeException</code> | Parameter out of range (e.g. DO port)     | SDK validation |
| <code>InvalidOperationException</code>   | Not connected                             | SDK state      |
| <code>ObjectDisposedException</code>     | Object already disposed                   | SDK state      |

## CodroidCommandException Properties

```
public class CodroidCommandException : Exception
{
    public int RequestId { get; } // Protocol request ID
    public string CommandType { get; } // e.g. "Robot/move"
    public string? ControllerError { get; } // err field from controller
    public CommonResponse? Response { get; } // Full response
}
```

## Thread Safety

- `CriData` -- Thread-safe (returns a clone)
- `Data` -- Not thread-safe (direct reference)
- All TCP methods -- Safe to call from any thread, but do not call concurrently on the same



CodroidClient

- CriRealtimeDispatcher -- SendCommand / SendTrajectory are not thread-safe

## 3. CodroidClient API Reference

**Class:** `CodroidClient`

**Namespace:** `Codroid`

**Source:** `CodroidSDK/Codroid.cs`

### Constructor

```
public CodroidClient(string ip)
```

| Parameter       | Type                | Description           |
|-----------------|---------------------|-----------------------|
| <code>ip</code> | <code>string</code> | Controller IP address |

TCP port is fixed at **9001**.

```
var robot = new CodroidClient("192.168.8.136");
```

### Properties

| Property             | Type                         | Description                                                       |
|----------------------|------------------------------|-------------------------------------------------------------------|
| <code>CriData</code> | <code>CriRealTimeData</code> | Thread-safe clone of CRI data snapshot                            |
| <code>Data</code>    | <code>CriRealTimeData</code> | Direct reference to internal CRI buffer (faster, not thread-safe) |

```
// Thread-safe (returns clone)
double[] joints = robot.CriData.JointPosition;

// Direct reference (faster)
double[] joints2 = robot.Data.JointPosition;
```

### Event

```
public event Action<CriRealTimeData>? CriDataReceived
```

Fires after each valid CRI UDP frame is parsed. The parameter is a thread-safe clone.

```
robot.CriDataReceived += data =>
{
    Console.WriteLine($"Joints: {string.Join(", ", data.JointPosition)}");
    Console.WriteLine($"TCP: {string.Join(", ", data.TcpPose)}");
    Console.WriteLine($"InMotion: {data.InMotion}");
};
```

---

# 1. Connection Management

## Connect

```
public async Task Connect()
```

Establishes TCP connection to the controller.

```
await robot.Connect();
```

---

## ConnectRemoteAndSwitchOn

```
public async Task ConnectRemoteAndSwitchOn()
```

Connects TCP, enters remote mode via auto, then powers on. This is the recommended one-call setup.

```
await robot.ConnectRemoteAndSwitchOn();
```

---

## Disconnect

```
public void Disconnect()
```

Stops CRI UDP listener and disconnects TCP. Always call in `finally`.

```
try
{
    await robot.ConnectRemoteAndSwitchOn();
    // ... operations ...
}
finally
{
    robot.Disconnect();
}
```

## 2. Mode Switching

### SwitchOn / SwitchOff

```
public async Task<CommonResponse> SwitchOn()  
public async Task<CommonResponse> SwitchOff()
```

Power on / power off the robot.

| Method      | Protocol        |
|-------------|-----------------|
| SwitchOn()  | Robot/switchOn  |
| SwitchOff() | Robot/switchOff |

```
await robot.SwitchOn();  
// ... operations ...  
await robot.SwitchOff();
```

### ToManual / ToAuto / ToRemote

```
public Task<CommonResponse> ToManual()  
public Task<CommonResponse> ToAuto()  
public Task<CommonResponse> ToRemote()
```

Switch to manual / auto / remote mode. Requires firmware 2.3.2.6+.

| Method     | Protocol       |
|------------|----------------|
| ToManual() | Robot/toManual |
| ToAuto()   | Robot/toAuto   |
| ToRemote() | Robot/toRemote |

### EnterManualModeViaAuto / EnterRemoteModeViaAuto

```
public async Task<CommonResponse> EnterManualModeViaAuto()  
public async Task<CommonResponse> EnterRemoteModeViaAuto()
```

Switches to auto first, then to manual / remote. Satisfies the controller's "must go through auto" restriction.

```
await robot.EnterRemoteModeViaAuto();
```

## ToSimulation / ToActual

```
public Task<CommonResponse> ToSimulation()  
public Task<CommonResponse> ToActual()
```

Switch to simulation / real-machine mode.

## StartDrag / StopDrag

```
public Task<CommonResponse> StartDrag()  
public Task<CommonResponse> StopDrag()
```

Enter / exit drag mode. Requires firmware 2.3.2.6+.

## ClearSystemError

```
public Task<CommonResponse> ClearSystemError()
```

Clears the system error state.

| Method             | Protocol          |
|--------------------|-------------------|
| ClearSystemError() | System/clearError |

## 3. Motion Commands (Non-Blocking)

All motion methods send the command and return immediately. Use `*Sync` variants for blocking wait.

### MovJ -- Joint Move

```
public Task<CommonResponse> MovJ(JointPoint target, double speed, double acc,  
    double blend = 25, double[]? coor = null, double[]? tool = null, double? relativeBlend =  
    null)  
  
public Task<CommonResponse> MovJ(CartesianPoint target, double speed, double acc,  
    double blend = 25, double[]? coor = null, double[]? tool = null, double? relativeBlend =  
    null)
```

| Parameter | Type                        | Default | Description     |
|-----------|-----------------------------|---------|-----------------|
| target    | JointPoint / CartesianPoint | --      | Target position |
| speed     | double                      | --      | Speed           |
| acc       | double                      | --      | Acceleration    |

| Parameter     | Type      | Default | Description           |
|---------------|-----------|---------|-----------------------|
| blend         | double    | 25      | Blend radius          |
| coor          | double[]? | null    | User coordinate frame |
| tool          | double[]? | null    | Tool coordinate frame |
| relativeBlend | double?   | null    | Relative blend        |

```
// Joint target
await robot.MovJ(JointPoint.Degrees(new[] { 0, 0, 90, 0, 90, 0 }), speed: 40, acc: 100);

// Cartesian target (joint motion to TCP pose)
await robot.MovJ(CartesianPoint.MmDeg(new[] { 400, 0, 300, 180, 0, 0 }), speed: 40, acc: 100);
```

## MovL -- Linear Move

```
public Task<CommonResponse> MovL(CartesianPoint target, double speed, double acc,
    double blend = 25, double[]? coor = null, double[]? tool = null, double? relativeBlend = null)

public Task<CommonResponse> MovL(JointPoint target, double speed, double acc,
    double blend = 25, double[]? coor = null, double[]? tool = null, double? relativeBlend = null)
```

```
// Cartesian linear move
await robot.MovL(CartesianPoint.MmDegWithRef(pose, robot.CriData.JointPosition),
    speed: 150, acc: 500);

// Linear move to joint target
await robot.MovL(JointPoint.Degrees(new[] { 10, 20, 90, 0, 90, 0 }), speed: 100, acc: 300);
```

## MovC -- Circular Move

```
public Task<CommonResponse> MovC(CartesianPoint middle, CartesianPoint target,
    double speed, double acc, double blend = 25,
    double[]? coor = null, double[]? tool = null, double? relativeBlend = null)
```

| Parameter | Type           | Description                 |
|-----------|----------------|-----------------------------|
| middle    | CartesianPoint | Intermediate point (on arc) |
| target    | CartesianPoint | End point                   |

```
await robot.MovC(
    CartesianPoint.MmDeg(new[] { 450, 100, 300, 180, 0, 0 }),
    CartesianPoint.MmDeg(new[] { 500, 0, 300, 180, 0, 0 }),
    speed: 100, acc: 300);
```

## MovCircle -- Full Circle Move

```
public Task<CommonResponse> MovCircle(CartesianPoint middle, CartesianPoint target,
    int circleNum, double speed, double acc, double blend = 25,
    double[]? coor = null, double[]? tool = null, double? relativeBlend = null)
```

| Parameter | Type           | Description            |
|-----------|----------------|------------------------|
| middle    | CartesianPoint | Intermediate point     |
| target    | CartesianPoint | End point              |
| circleNum | int            | Number of full circles |

```
await robot.MovCircle(
    CartesianPoint.MmDeg(mid),
    CartesianPoint.MmDeg(end),
    circleNum: 1, speed: 80, acc: 200);
```

## Move -- Multi-Segment Path

```
public async Task<CommonResponse> Move(IReadOnlyList<MoveInstruction> instructions)
```

Sends a list of motion instructions as a single path command.

```
await robot.Move(new[]
{
    MoveInstruction.MovJ(JointPoint.Degrees(new[] { 0, 0, 90, 0, 90, 0 }), 40, 100),
    MoveInstruction.MovL(CartesianPoint.MmDegWithRef(pose, robot.CriData.JointPosition),
    150, 500),
    MoveInstruction.MovC(CartesianPoint.MmDeg(mid), CartesianPoint.MmDeg(end), 100, 300),
});
```

## 4. Blocking Motion Commands

\*Sync methods send the motion command, then **block until CRI confirms the robot has reached the target**. They return `true` on success, or throw on error/timeout.

**Prerequisite:** `StartCriDataPush` must be active.

## MovJSync

```
public bool MovJSync(JointPoint target, double speed, double acc,  
    MotionWaitOptions? wait = null, double blend = 25,  
    double[]? coor = null, double[]? tool = null, double? relativeBlend = null)
```

```
public bool MovJSync(CartesianPoint target, double speed, double acc,  
    MotionWaitOptions? wait = null, double blend = 25,  
    double[]? coor = null, double[]? tool = null, double? relativeBlend = null)
```

```
var wait = new MotionWaitOptions  
{  
    Timeout = TimeSpan.FromSeconds(90),  
    JointToleranceDeg = 0.3  
};  
  
robot.MovJSync(JointPoint.Degrees(new[] { 0, 0, 90, 0, 90, 0 }), 40, 100, wait);
```

---

## MovLSync

```
public bool MovLSync(CartesianPoint target, double speed, double acc,  
    MotionWaitOptions? wait = null, double blend = 25,  
    double[]? coor = null, double[]? tool = null, double? relativeBlend = null)
```

```
public bool MovLSync(JointPoint target, double speed, double acc,  
    MotionWaitOptions? wait = null, double blend = 25,  
    double[]? coor = null, double[]? tool = null, double? relativeBlend = null)
```

```
robot.MovLSync(  
    CartesianPoint.MmDegWithRef(pose, robot.CriData.JointPosition),  
    speed: 150, acc: 500,  
    wait: new MotionWaitOptions  
    {  
        Timeout = TimeSpan.FromSeconds(60),  
        CartesianPositionToleranceMm = 2.0,  
        CartesianOrientationToleranceDeg = 1.5  
    });
```

---

## MovCSync

```
public bool MovCSync(CartesianPoint middle, CartesianPoint target,  
    double speed, double acc, MotionWaitOptions? wait = null,  
    double blend = 25, double[]? coor = null, double[]? tool = null, double? relativeBlend =  
    null)
```



```
robot.MovCSync(  
    CartesianPoint.MmDeg(mid), CartesianPoint.MmDeg(end),  
    speed: 100, acc: 300);
```

---

## MovCircleSync

```
public bool MovCircleSync(CartesianPoint middle, CartesianPoint target,  
    int circleNum, double speed, double acc, MotionWaitOptions? wait = null,  
    double blend = 25, double[]? coor = null, double[]? tool = null, double? relativeBlend =  
    null)
```

---

## MoveSync

```
public bool MoveSync(IReadOnlyList<MoveInstruction> instructions, MotionWaitOptions? wait =  
    null)
```

Sends multi-segment path and blocks until the last segment target is reached.

```
robot.MoveSync(new[]  
{  
    MoveInstruction.MovJ(JointPoint.Degrees(j1), 40, 100),  
    MoveInstruction.MovL(CartesianPoint.MmDegWithRef(p2, refJ), 150, 500),  
});
```

---

## 5. Motion Control

### PauseRobotMotion

```
public async Task<CommonResponse> PauseRobotMotion()
```

Pauses the current motion.

| Method             | Protocol    |
|--------------------|-------------|
| PauseRobotMotion() | Robot/pause |

---

### ResumeRobotMotion

```
public async Task<CommonResponse> ResumeRobotMotion()
```

Resumes paused motion.

| Method                           | Protocol                  |
|----------------------------------|---------------------------|
| <code>ResumeRobotMotion()</code> | <code>Robot/resume</code> |

## StopRobotMove

```
public async Task<CommonResponse> StopRobotMove()
```

Stops the current motion immediately.

| Method                       | Protocol                    |
|------------------------------|-----------------------------|
| <code>StopRobotMove()</code> | <code>Robot/stopMove</code> |

## 6. MoveTo Commands

### MoveTo

```
public async Task<CommonResponse> MoveTo(MoveToKind kind, MoveToTarget? target = null)
```

Moves to a preset or planned position. Requires heartbeat while running.

| Method                            | Protocol                  |
|-----------------------------------|---------------------------|
| <code>MoveTo(kind, target)</code> | <code>Robot/moveTo</code> |

```
// Move to home position
await robot.MoveTo(MoveToKind.Home);

// Move to safe position
await robot.MoveTo(MoveToKind.Safe);

// Move to specific joint position
await robot.MoveTo(MoveToKind.JointPlanned, MoveToTarget.Joint(JointPoint.Degrees(joints)));
```

### MoveToHeartbeat

```
public async Task<CommonResponse> MoveToHeartbeat()
```

Sends heartbeat to maintain MoveTo motion. Call at ~500ms intervals.

| Method                         | Protocol                           |
|--------------------------------|------------------------------------|
| <code>MoveToHeartbeat()</code> | <code>Robot/moveToHeartbeat</code> |

---

## StopMoveTo

```
public async Task<CommonResponse> StopMoveTo()
```

Stops the current MoveTo / RunTo motion.

| Method       | Protocol               |
|--------------|------------------------|
| StopMoveTo() | Robot/moveTo (type=-1) |

---

## 7. Jog Commands

### StartJog

```
public async Task<CommonResponse> StartJog(RobotJogParameters parameters)
```

Starts jogging. Requires heartbeat at ~500ms intervals.

| Method               | Protocol  |
|----------------------|-----------|
| StartJog(parameters) | Robot/jog |

```
var jogParams = RobotJogParameters.Create(  
    RobotJogMode.Joint,    // Joint mode  
    speed: 10,             // Speed  
    index: 0,              // Axis index (0-5)  
    RobotJogFrameType.User, // User frame  
    coorId: 0              // Coordinate ID  
);  
  
await robot.StartJog(jogParams);  
  
// Keep sending heartbeat  
while (jogging)  
{  
    await Task.Delay(500);  
    await robot.JogHeartbeat();  
}  
  
await robot.StopJog();
```

---

## StopJog

```
public async Task<CommonResponse> StopJog()
```

Stops jogging.

| Method    | Protocol      |
|-----------|---------------|
| StopJog() | Robot/stopJog |

## JogHeartbeat

```
public async Task<CommonResponse> JogHeartbeat()
```

Sends heartbeat to maintain jog state. Call at ~500ms intervals.

| Method         | Protocol           |
|----------------|--------------------|
| JogHeartbeat() | Robot/jogHeartbeat |

## 8. IO Operations

### GetDi / GetDo / GetAi / GetAo

```
public async Task<int> GetDi(int port) // Read DI
public async Task<int> GetDo(int port) // Read DO
public async Task<double> GetAi(int port) // Read AI
public async Task<double> GetAo(int port) // Read AO
```

| Method      | Returns      | Protocol             |
|-------------|--------------|----------------------|
| GetDi(port) | int (0 or 1) | IOManager/GetIOValue |
| GetDo(port) | int (0 or 1) | IOManager/GetIOValue |
| GetAi(port) | double       | IOManager/GetIOValue |
| GetAo(port) | double       | IOManager/GetIOValue |

```
int di0 = await robot.GetDi(0);
Console.WriteLine($"DI 0 = {di0}");

double ai1 = await robot.GetAi(1);
Console.WriteLine($"AI 1 = {ai1:F3}");
```

## SetDo / SetAo

```
public async Task<CommonResponse> SetDo(int port, int value)    // Write DO (0 or 1)
public async Task<CommonResponse> SetAo(int port, double value) // Write AO
```

| Method             | Protocol             |
|--------------------|----------------------|
| SetDo(port, value) | IOManager/SetIOValue |
| SetAo(port, value) | IOManager/SetIOValue |

```
await robot.SetDo(10, 1); // Set DO 10 high
await robot.SetDo(10, 0); // Set DO 10 low
await robot.SetAo(0, 3.14); // Set AO 0 to 3.14
```

## GetIoValues (Batch Read)

```
public async Task<CommonResponse> GetIoValues(IReadOnlyList<(string Type, int Port)> pins)
```

```
var pins = new (string Type, int Port)[]
{
    ("DI", 0), ("DI", 1), ("DO", 10), ("AI", 0)
};

CommonResponse resp = await robot.GetIoValues(pins);
// Results in resp.db
```

## 9. Register Operations

### GetRegisterValue

```
public async Task<RegisterReadValue> GetRegisterValue(int address)
```

```
RegisterReadValue reg = await robot.GetRegisterValue(49100);
int intVal = reg.GetInt32();
double dblVal = reg.GetDouble();
```

### GetRegisterValues (Batch Read)

```
public async Task<IReadOnlyList<RegisterReadValue>> GetRegisterValues(IReadOnlyList<int>
addresses)
```

```
var addresses = new[] { 49100, 49101, 49102 };
IReadOnlyList<RegisterReadValue> values = await robot.GetRegisterValues(addresses);

for (int i = 0; i < values.Count; i++)
    Console.WriteLine($"Register {addresses[i]} = {values[i].GetInt32()}");
```

---

## SetRegisterValue

```
public async Task<CommonResponse> SetRegisterValue(int address, int value)
public async Task<CommonResponse> SetRegisterValue(int address, double value)
```

```
await robot.SetRegisterValue(49100, 42);
await robot.SetRegisterValue(49101, 3.14);
```

---

## SetExtendArrayType / RemoveExtendArray

```
public async Task<CommonResponse> SetExtendArrayType(int index, string type)
public async Task<CommonResponse> RemoveExtendArray(int index)
```

Manage extend-array elements (index 0~999).

```
await robot.SetExtendArrayType(0, RegisterExtendArrayValueType.Int32);
await robot.RemoveExtendArray(0);
```

---

## 10. Robot Settings (19.x Protocol)

### SetManualMoveRate / SetAutoMoveRate

```
public async Task<CommonResponse> SetManualMoveRate(int percent)
public async Task<CommonResponse> SetAutoMoveRate(int percent)
```

Set manual / auto motion rate (1~100%).

```
await robot.SetManualMoveRate(50); // 50% speed
await robot.SetAutoMoveRate(100); // Full speed
```

---

### SetCollisionSensitivity

```
public async Task<CommonResponse> SetCollisionSensitivity(int sensitivity)
```

Set collision detection sensitivity (0~100). Firmware 2.3.2.10+.

```
await robot.SetCollisionSensitivity(50);
```

---

## SetPayload

```
public async Task<CommonResponse> SetPayload(int payloadId)
```

Set active payload slot (0~15). Firmware 2.3.2.10+.

```
await robot.SetPayload(1); // Use payload slot 1
```

---

## GetRobotParameters

```
public async Task<RobotParameters> GetRobotParameters()
```

Gets all setting-interface parameters (protocol 19.7). Returns tool frames, payload frames, coordinate frames, and default IDs.

```
RobotParameters param = await robot.GetRobotParameters();  
Console.WriteLine($"Default Tool: {param.DefaultToolId}");  
Console.WriteLine($"Default Payload: {param.DefaultPayloadId}");  
Console.WriteLine($"Max Payload: {param.MaxPayload} kg");
```

---

## SetDefaultPayloadId / SetDefaultToolId / SetDefaultUserCoordinateId

```
public Task<CommonResponse> SetDefaultPayloadId(int payloadId) // 0~15  
public Task<CommonResponse> SetDefaultToolId(int toolId) // 0~15  
public Task<CommonResponse> SetDefaultUserCoordinateId(int coordinateId) // 0~15
```

Set default payload / tool / user coordinate frame slot.

```
await robot.SetDefaultToolId(2);  
await robot.SetDefaultPayloadId(1);  
await robot.SetDefaultUserCoordinateId(0);
```

---

## SaveToolFrames / SetToolFrame

```
public Task<CommonResponse> SaveToolFrames(IReadOnlyList<RobotFrame> frames)  
public async Task<CommonResponse> SetToolFrame(int frameId, RobotFrame frame)
```

Save the full tool frame table (must include id 0~15, id=0 must be all zeros) / Modify a single tool frame (read-then-write, id 1~15 only).

```
// Set single tool frame
await robot.SetToolFrame(1, new RobotFrame
{
    Id = 1, X = 0, Y = 0, Z = 100, A = 0, B = 0, C = 0
});
```

## SavePayloadFrames / SetPayloadFrame

```
public Task<CommonResponse> SavePayloadFrames(IReadOnlyList<RobotPayloadFrame> frames)
public async Task<CommonResponse> SetPayloadFrame(int frameId, RobotPayloadFrame frame)
```

Save full payload frame table / Modify single payload frame (id 1~15).

```
await robot.SetPayloadFrame(1, new RobotPayloadFrame
{
    Id = 1, M = 2.5, Mx = 0, My = 0, Mz = 50
});
```

## SaveUserCoordinateFrames / SetUserCoordinateFrame

```
public Task<CommonResponse> SaveUserCoordinateFrames(IReadOnlyList<RobotFrame> frames)
public async Task<CommonResponse> SetUserCoordinateFrame(int frameId, RobotFrame frame)
```

Save full user coordinate frame table / Modify single user coordinate frame (id 1~15).

```
await robot.SetUserCoordinateFrame(1, new RobotFrame
{
    Id = 1, X = 100, Y = 200, Z = 0, A = 0, B = 0, C = 45
});
```

# 11. CRI Real-Time Data

## StartCriDataPush

```
public async Task<CommonResponse> StartCriDataPush(string udpIp, int udpPort)
```

Starts local UDP listener and requests controller to push CRI real-time data. Fixed params: 100ms period, high-precision, mask 0xFFFF, 308-byte UDP packet.

| Method                           | Protocol          |
|----------------------------------|-------------------|
| StartCriDataPush(udpIp, udpPort) | CRI/StartDataPush |



```
await robot.StartCriDataPush("192.168.8.150", 18888);

robot.CriDataReceived += data =>
{
    Console.WriteLine($"Joints: {string.Join(", ", data.JointPosition)}");
};
```

## StopCriDataPush

```
public async Task<CommonResponse> StopCriDataPush(string? udpIp = null, int? udpPort = null)
```

Requests controller to stop CRI data push and closes local UDP listener.

| Method                    | Protocol         |
|---------------------------|------------------|
| StopCriDataPush(ip, port) | CRI/StopDataPush |

```
await robot.StopCriDataPush("192.168.8.150", 18888);
```

## 12. CRI Real-Time Control

### StartCriControl

```
public async Task<CommonResponse> StartCriControl(int filterType = 1, int durationMs = 4,
int startBuffer = 5)
```

Enables CRI real-time control mode.

| Parameter   | Type | Default | Description                                  |
|-------------|------|---------|----------------------------------------------|
| filterType  | int  | 1       | 0=off, 1=average, 2=2nd-order LP, 3=elliptic |
| durationMs  | int  | 4       | Control period (1~16ms, must divide 1000)    |
| startBuffer | int  | 5       | Start buffer frames (1~100)                  |

| Method               | Protocol         |
|----------------------|------------------|
| StartCriControl(...) | CRI/StartControl |

```
await robot.StartCriControl(filterType: 1, durationMs: 4, startBuffer: 5);
```

## StopCriControl

```
public async Task<CommonResponse> StopCriControl()
```

Disables CRI real-time control mode.

| Method           | Protocol        |
|------------------|-----------------|
| StopCriControl() | CRI/StopControl |

## 13. Project Execution

### EnterRemoteScriptMode

```
public async Task<CommonResponse> EnterRemoteScriptMode()
```

Requests entering remote script mode.

| Method                  | Protocol                      |
|-------------------------|-------------------------------|
| EnterRemoteScriptMode() | project/enterRemoteScriptMode |

### RunScript

```
public async Task<CommonResponse> RunScript(  
    string mainScript,  
    IReadOnlyDictionary<string, string>? subThreads = null,  
    IReadOnlyDictionary<string, string>? subPrograms = null,  
    IReadOnlyDictionary<string, string>? interrupts = null,  
    IReadOnlyDictionary<string, object>? vars = null)
```

Sends a script for immediate execution.

```
await robot.RunScript(mainScript: "movej(j1, v50) sub1() end");
```

### Run / RunByIndex / RunStep

```
public async Task<CommonResponse> Run(string projectID)  
public async Task<CommonResponse> RunByIndex(int index)  
public async Task<CommonResponse> RunStep(string projectID)
```

Start a project by ID / index / single-step.

```
await robot.Run("project_001");
await robot.RunByIndex(0);
await robot.RunStep("project_001");
```

---

## PauseProject / ResumeProject / StopProject

```
public async Task<CommonResponse> PauseProject()
public async Task<CommonResponse> ResumeProject()
public async Task<CommonResponse> StopProject()
```

| Method          | Protocol       |
|-----------------|----------------|
| PauseProject()  | project/pause  |
| ResumeProject() | project/resume |
| StopProject()   | project/stop   |

---

## 14. Publish/Subscribe

### SubscribePublishTopic

```
public async Task<PublishTopicSubscription> SubscribePublishTopic(
    string topicTy, Action<PublishNotification> handler, int tcMilliseconds = 100)
```

Subscribes to a TCP topic push. Returns a disposable subscription handle.

```
using var sub = await robot.SubscribePublishTopic(
    PublishTopics.RobotStatus,
    notification =>
    {
        Console.WriteLine($"Topic: {notification.Ty}");
        Console.WriteLine($"Data: {notification.Db}");
    });

// Subscription active until sub.Dispose()
await Task.Delay(10000);
// sub.Dispose() called by 'using'
```

## 15. Global Variables

---

### GetGlobalVars / GetGlobalVarsCatalog

```
public async Task<CommonResponse> GetGlobalVars()  
public async Task<IReadOnlyDictionary<string, GlobalVarCatalogEntry>> GetGlobalVarsCatalog()
```

```
var catalog = await robot.GetGlobalVarsCatalog();  
foreach (var (name, entry) in catalog)  
{  
    Console.WriteLine($"{name} = {entry.Value} ({entry.Remark})");  
}
```

---

### SaveGlobalVar / SaveGlobalVars

```
public Task<CommonResponse> SaveGlobalVar(string name, object value, string? remark = null)  
public async Task<CommonResponse> SaveGlobalVars(IReadOnlyCollection<GlobalVarSaveItem>  
items)
```

```
// Single  
await robot.SaveGlobalVar("counter", 42, "test counter");  
  
// Batch  
await robot.SaveGlobalVars(new[]  
{  
    new GlobalVarSaveItem("x", 100.0, "X position"),  
    new GlobalVarSaveItem("y", 200.0, "Y position"),  
});
```

---

### RemoveGlobalVars

```
public async Task<CommonResponse> RemoveGlobalVars(IEnumerable<string> names)
```

Deletes specified global variables. Deleting nonexistent variables is not an error.

```
await robot.RemoveGlobalVars(new[] { "counter", "x", "y" });
```

---

## 16. Kinematics

### AposToCpos / AposToCposPose (Forward Kinematics)

```
public async Task<CommonResponse> AposToCpos(double[] jointDegrees, double[] userFrame,
double[] toolFrame, double[]? externalAxisPositions = null)
public async Task<double[]> AposToCposPose(double[] jointDegrees, double[] userFrame,
double[] toolFrame, double[]? externalAxisPositions = null)
```

Forward kinematics: joint space to Cartesian space. `AposToCposPose` returns [x,y,z,rx,ry,rz] in mm+deg.

```
double[] joints = { 0, 0, 90, 0, 90, 0 };
double[] userFrame = { 0, 0, 0, 0, 0, 0 };
double[] toolFrame = { 0, 0, 100, 0, 0, 0 };

double[] pose = await robot.AposToCposPose(joints, userFrame, toolFrame);
Console.WriteLine($"TCP: [{string.Join(", ", pose.Select(v => v.ToString("F1"))}]");
// TCP: [400.0, 0.0, 300.0, 180.0, 0.0, 0.0]
```

### CposToApos / CposToAposJoints (Inverse Kinematics)

```
public async Task<CommonResponse> CposToApos(double[] cartesianMmDeg, double[]
referenceJointDegrees, double[]? externalAxisPositions = null)
public async Task<double[]> CposToAposJoints(double[] cartesianMmDeg, double[]
referenceJointDegrees, double[]? externalAxisPositions = null)
```

Inverse kinematics: Cartesian to joint space. `CposToAposJoints` returns 6 joint angles in degrees.

```
double[] pose = { 400, 0, 300, 180, 0, 0 };
double[] refJoints = { 0, 0, 90, 0, 90, 0 };

double[] joints = await robot.CposToAposJoints(pose, refJoints);
Console.WriteLine($"Joints: [{string.Join(", ", joints.Select(v => v.ToString("F2"))}]");
```

### CalculateRelativePose / CalculateRelativePoseResult

```
public async Task<CommonResponse> CalculateRelativePose(double[] tcpPoseWorld, double[]
offset, RelativePoseCoorType coorType, double[]? tcpPoseInPosCoorFrame = null, double[]?
userCoorFrame = null)
public async Task<double[]> CalculateRelativePoseResult(double[] tcpPoseWorld, double[]
offset, RelativePoseCoorType coorType, double[]? tcpPoseInPosCoorFrame = null, double[]?
userCoorFrame = null)
```

Calculates a relative pose / offset in user or tool coordinate frame.

| Parameter                           | Type                               | Description                           |
|-------------------------------------|------------------------------------|---------------------------------------|
| <code>tcpPoseWorld</code>           | <code>double[]</code>              | Current TCP pose in world frame       |
| <code>offset</code>                 | <code>double[]</code>              | [dx,dy,dz,drx,dry,drz] offset         |
| <code>coordType</code>              | <code>RelativePoseCoordType</code> | User or Tool                          |
| <code>tcpPoseInPosCoordFrame</code> | <code>double[]?</code>             | TCP pose in position coordinate frame |
| <code>userCoordFrame</code>         | <code>double[]?</code>             | User coordinate frame definition      |

```
double[] currentPose = { 400, 0, 300, 180, 0, 0 };
double[] offset = { 50, 0, 0, 0, 0, 0 }; // Move +50mm in X

double[] newPose = await robot.CalculateRelativePoseResult(
    currentPose, offset, RelativePoseCoordType.User);

Console.WriteLine($"New pose: [{string.Join(", ", newPose)}]");
```

# 4. Motion API Reference

This section covers all motion-related types in the CodroidCS SDK, including joint/Cartesian point definitions, move instructions, jog parameters, and motion wait options.

## Table of Contents

- 1. [JointPoint](#)
- 2. [CartesianPoint](#)
- 3. [MovePoint](#)
- 4. [MoveInstruction](#)
- 5. [MoveToTarget](#)
- 6. [MoveToKind](#)
- 7. [MoveKinds](#)
- 8. [MotionWaitOptions](#)
- 9. [RobotJogParameters](#)
- 10. [RobotJogMode](#)
- 11. [RobotJogFrameType](#)

## 1. JointPoint

A sealed class representing a robot target defined by joint angles.

`JointPoint` stores 6 joint angles in degrees and is used when you want to move the robot to an exact joint configuration without ambiguity.

### Properties

| Property        | Type                  | Description               |
|-----------------|-----------------------|---------------------------|
| <code>Jp</code> | <code>double[]</code> | 6 joint angles in degrees |

### Factory Methods

| Method                                              | Description                                                                       |
|-----------------------------------------------------|-----------------------------------------------------------------------------------|
| <code>JointPoint.Degrees(double[] jointsDeg)</code> | Create from 6 joint angles in degrees. The array <b>must</b> be exactly length 6. |

# Example

```
// Create a joint point with all 6 joint angles in degrees
var jp = JointPoint.Degrees(new[] { 0, 0, 90, 0, 90, 0 });

// Use in a move instruction
await robot.Move(MoveInstruction.MovJ(jp, speed: 40, acc: 100));
```

## 2. CartesianPoint

A sealed class representing a robot target defined by Cartesian (TCP) pose, with optional reference joints for inverse kinematics.

`CartesianPoint` stores a TCP pose as `[x, y, z, rx, ry, rz]` in millimeters and degrees. When only a pose is provided, the controller uses default reference joints `[20, 20, 20, 20, 20, 20]` for inverse kinematics. You can supply explicit reference joints to guide the IK solver toward a specific configuration.

### Properties

| Property        | Type                   | Description                                                                                                               |
|-----------------|------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <code>Cp</code> | <code>double[]</code>  | TCP pose <code>[x, y, z, rx, ry, rz]</code> -- position in mm, orientation in degrees                                     |
| <code>Rj</code> | <code>double[]?</code> | Reference joints for IK (6 joint angles in degrees). <code>null</code> uses default <code>[20, 20, 20, 20, 20, 20]</code> |

### Factory Methods

| Method                                                                              | Description                                               |
|-------------------------------------------------------------------------------------|-----------------------------------------------------------|
| <code>CartesianPoint.MmDeg(double[] poseMmDeg)</code>                               | Create with TCP pose only (uses default reference joints) |
| <code>CartesianPoint.MmDegWithRef(double[] poseMmDeg, double[] refJointsDeg)</code> | Create with TCP pose and explicit reference joints for IK |



## Examples

```
// Create a Cartesian point with TCP pose only (position + orientation)
var cp = CartesianPoint.MmDeg(new[] { 400, 0, 300, 180, 0, 0 });

// Create a Cartesian point with explicit reference joints from current robot state
var refJ = robot.CriData.JointPosition;
var cpWithRef = CartesianPoint.MmDegWithRef(
    new[] { 400, 0, 300, 180, 0, 0 },
    refJ
);

// Use in a linear move instruction
await robot.Move(MoveInstruction.MovL(cp, speed: 150, acc: 500));
```

## 3. MovePoint

A sealed class used internally for serialization of move target points.

`MovePoint` is the serialization wrapper used when sending move instructions to the controller. It holds the optional joint (`Jp`), Cartesian (`Cp`), reference joint (`Rj`), and external (`Ep`) arrays. You typically do not create `MovePoint` instances directly; use the factory methods on `MoveInstruction` instead.

### Properties

| Property        | Type                   | Description                                      |
|-----------------|------------------------|--------------------------------------------------|
| <code>Jp</code> | <code>double[]?</code> | Joint angles (degrees), null if Cartesian target |
| <code>Cp</code> | <code>double[]?</code> | TCP pose (mm + deg), null if joint target        |
| <code>Rj</code> | <code>double[]?</code> | Reference joints for IK                          |
| <code>Ep</code> | <code>double[]?</code> | External axes                                    |

All properties use `[JsonIgnoreWhenNull]` -- they are omitted from JSON serialization when null.

### Factory Methods

| Method                                                  | Description                               |
|---------------------------------------------------------|-------------------------------------------|
| <code>MovePoint.FromJoint(JointPoint jp)</code>         | Create from a <code>JointPoint</code>     |
| <code>MovePoint.FromCartesian(CartesianPoint cp)</code> | Create from a <code>CartesianPoint</code> |

## Example

```
// Typically you do not create MovePoint directly. Use MoveInstruction factories.

// If you need to wrap a point explicitly:
var jp = JointPoint.Degrees(new[] { 0, 0, 90, 0, 90, 0 });
var movePoint = MovePoint.FromJoint(jp);

var cp = CartesianPoint.MmDeg(new[] { 400, 0, 300, 180, 0, 0 });
var movePointFromCart = MovePoint.FromCartesian(cp);
```

## 4. MoveInstruction

A sealed class that defines a single motion segment in a robot move command.

`MoveInstruction` is the primary type for building motion paths. Each instance describes one segment with a motion type (joint, linear, or circular), speed/acceleration parameters, blending settings, and optional coordinate system and tool offsets.

### Properties

| Property      | Type       | Default | Description                                                                   |
|---------------|------------|---------|-------------------------------------------------------------------------------|
| Type          | string     | "movJ"  | Motion type: "movJ", "movL", "movC", "movCircle"                              |
| CircleNum     | int?       | null    | Number of full circles (only for <code>movCircle</code> )                     |
| Speed         | double     | --      | Speed value (mm/s for linear, deg/s for joint)                                |
| Acc           | double     | --      | Acceleration value                                                            |
| Blend         | double     | --      | Blend radius (mm for linear, deg for joint)                                   |
| RelativeBlend | double?    | null    | Relative blend ratio (0-1), overrides <code>Blend</code> when set             |
| TargetPoint   | MovePoint  | --      | The target point for this segment                                             |
| MiddlePoint   | MovePoint? | null    | Middle/via point (required for <code>movC</code> and <code>movCircle</code> ) |
| Coor          | double[]?  | null    | Coordinate system definition                                                  |
| Tool          | double[]?  | null    | Tool definition                                                               |

### Factory Methods

All factories share common optional parameters: `coor` (coordinate system), `tool` (tool offset), and `relativeBlend` (relative blend ratio).

| Method                                                                                                                      | Motion Type | Target Types                  | Description                           |
|-----------------------------------------------------------------------------------------------------------------------------|-------------|-------------------------------|---------------------------------------|
| <code>MoveInstruction.MovJ(JointPoint, speed, acc, blend, ...)</code>                                                       | Joint       | JointPoint                    | Joint move to joint target            |
| <code>MoveInstruction.MovJ(CartesianPoint, speed, acc, blend, ...)</code>                                                   | Joint       | CartesianPoint                | Joint move to Cartesian target        |
| <code>MoveInstruction.MovL(CartesianPoint, speed, acc, blend, ...)</code>                                                   | Linear      | CartesianPoint                | Linear move to Cartesian target       |
| <code>MoveInstruction.MovL(JointPoint, speed, acc, blend, ...)</code>                                                       | Linear      | JointPoint                    | Linear move to joint target           |
| <code>MoveInstruction.MovC(CartesianPoint middle, CartesianPoint target, speed, acc, blend, ...)</code>                     | Circular    | 2x CartesianPoint             | Circular arc through middle to target |
| <code>MoveInstruction.MovCircle(CartesianPoint middle, CartesianPoint target, int circleNum, speed, acc, blend, ...)</code> | Full Circle | 2x CartesianPoint + circleNum | Full circle motion                    |

## Parameter Reference

| Parameter                  | Type                   | Default           | Description                     |
|----------------------------|------------------------|-------------------|---------------------------------|
| <code>speed</code>         | <code>double</code>    | --                | Required. Speed (mm/s or deg/s) |
| <code>acc</code>           | <code>double</code>    | --                | Required. Acceleration          |
| <code>blend</code>         | <code>double</code>    | <code>25</code>   | Blend radius                    |
| <code>coor</code>          | <code>double[]?</code> | <code>null</code> | Coordinate system               |
| <code>tool</code>          | <code>double[]?</code> | <code>null</code> | Tool offset                     |
| <code>relativeBlend</code> | <code>double?</code>   | <code>null</code> | Relative blend (0-1)            |

## Examples

```
// Single joint move to a joint target
var j1 = JointPoint.Degrees(new[] { 20, 20, 90, 0, 45, 0 });
await robot.Move(MoveInstruction.MovJ(j1, speed: 40, acc: 100));

// Single linear move to a Cartesian target
var cp = CartesianPoint.MmDeg(new[] { 400, 0, 300, 180, 0, 0 });
await robot.Move(MoveInstruction.MovL(cp, speed: 150, acc: 500));

// Multi-segment path: joint move followed by linear move
var p2 = new[] { 500, 100, 400, 180, 0, 0 };
var refJ = robot.CriData.JointPosition;
await robot.Move(new[]
```

```

{
    MoveInstruction.MovJ(JointPoint.Degrees(j1), 40, 100),
    MoveInstruction.MovL(CartesianPoint.MmDegWithRef(p2, refJ), 150, 500),
});

// Circular arc motion
var mid = CartesianPoint.MmDeg(new[] { 450, 50, 350, 180, 0, 0 });
var end = CartesianPoint.MmDeg(new[] { 500, 0, 300, 180, 0, 0 });
await robot.Move(MoveInstruction.MovC(mid, end, speed: 100, acc: 300));

// Full circle motion (2 full rotations)
await robot.Move(MoveInstruction.MovCircle(mid, end, circleNum: 2, speed: 80, acc: 200));

// With custom blend and coordinate system
await robot.Move(MoveInstruction.MovL(cp, speed: 100, acc: 300, blend: 10, coord:
userCoord));

```

## 5. MoveToTarget

A sealed class representing a target for pre-defined move-to commands (home, safe, pack, etc.).

`MoveToTarget` wraps a target point for use with `MoveToKind` commands. It can represent a joint target, a Cartesian target, or raw external axis data.

### Properties

| Property        | Type                   | Description                                                  |
|-----------------|------------------------|--------------------------------------------------------------|
| <code>Cp</code> | <code>double[]?</code> | Cartesian pose <code>[x, y, z, rx, ry, rz]</code> (mm + deg) |
| <code>Jp</code> | <code>double[]?</code> | Joint angles (degrees)                                       |
| <code>Ep</code> | <code>double[]?</code> | External axes                                                |

### Factory Methods

| Method                                                 | Description                               |
|--------------------------------------------------------|-------------------------------------------|
| <code>MoveToTarget.Joint(JointPoint jp)</code>         | Create from a <code>JointPoint</code>     |
| <code>MoveToTarget.Cartesian(CartesianPoint cp)</code> | Create from a <code>CartesianPoint</code> |

## Example

```
// Create a moveTo target from a joint point
var homeTarget = MoveToTarget.Joint(JointPoint.Degrees(new[] { 0, 0, 0, 0, 0, 0 }));

// Create a moveTo target from a Cartesian point
var safeTarget = MoveToTarget.Cartesian(
    CartesianPoint.MmDeg(new[] { 300, 0, 400, 180, 0, 0 })
);
```

## 6. MoveToKind

An enum specifying pre-defined move-to target types.

`MoveToKind` is used with the robot's `MoveTo` method to command the robot to move to well-known positions or resume program execution.

### Values

| Name          | Value | Description                   |
|---------------|-------|-------------------------------|
| Stop          | -1    | Stop the moveTo operation     |
| Home          | 0     | Home position                 |
| Safe          | 1     | Safe position                 |
| Candle        | 2     | Candle (vertical) position    |
| Pack          | 3     | Pack (transport) position     |
| JointPlanned  | 4     | Joint planned move to target  |
| LinePlanned   | 5     | Linear planned move to target |
| ProgramResume | 6     | Resume program execution      |

## Example

```
// Move the robot to the home position
await robot.MoveTo(MoveToKind.Home);

// Stop an in-progress moveTo operation
await robot.MoveTo(MoveToKind.Stop);

// Move to a specific joint position with joint planning
var target = MoveToTarget.Joint(JointPoint.Degrees(new[] { 10, 20, 90, 0, 45, 0 }));
await robot.MoveTo(MoveToKind.JointPlanned, target);

// Move to a Cartesian position with linear planning
var cartTarget = MoveToTarget.Cartesian(
```

```

        CartesianPoint.MmDeg(new[] { 400, 0, 300, 180, 0, 0 })
    );
    await robot.MoveTo(MoveToKind.LinePlanned, cartTarget);

    // Resume a paused program
    await robot.MoveTo(MoveToKind.ProgramResume);

```

## 7. MoveKinds

A static class providing string constants for motion type identifiers.

`MoveKinds` defines the string constants used in `MoveInstruction.Type`. These correspond to the four supported motion modes in the CodroidCS controller.

### Constants

| Name                   | Value                    | Description       |
|------------------------|--------------------------|-------------------|
| <code>MovJ</code>      | <code>"movJ"</code>      | Joint move        |
| <code>MovL</code>      | <code>"movL"</code>      | Linear move       |
| <code>MovC</code>      | <code>"movC"</code>      | Circular arc move |
| <code>MovCircle</code> | <code>"movCircle"</code> | Full circle move  |

### Example

```

// Compare the motion type of an instruction
var instruction = MoveInstruction.MovJ(
    JointPoint.Degrees(new[] { 0, 0, 90, 0, 90, 0 }),
    speed: 40, acc: 100
);

if (instruction.Type == MoveKinds.MovJ)
{
    Console.WriteLine("This is a joint move.");
}
else if (instruction.Type == MoveKinds.MovL)
{
    Console.WriteLine("This is a linear move.");
}

```

## 8. MotionWaitOptions

A sealed class that configures how the SDK waits for motion completion.

`MotionWaitOptions` controls the polling behavior, tolerance thresholds, and timeout when waiting for a robot motion to complete. You can customize these parameters to suit different precision and responsiveness requirements.

## Properties

| Property                                      | Type                  | Default    | Description                                                                  |
|-----------------------------------------------|-----------------------|------------|------------------------------------------------------------------------------|
| <code>Timeout</code>                          | <code>TimeSpan</code> | 60 seconds | Maximum time to wait for motion completion                                   |
| <code>PollInterval</code>                     | <code>TimeSpan</code> | 50 ms      | Interval between each poll to check motion status                            |
| <code>CriStaleTimeout</code>                  | <code>TimeSpan</code> | 500 ms     | Maximum age of CRI data before considered stale                              |
| <code>SettledSamples</code>                   | <code>int</code>      | 3          | Number of consecutive settled samples required to confirm motion is complete |
| <code>JointToleranceDeg</code>                | <code>double</code>   | 0.2        | Joint position tolerance in degrees                                          |
| <code>CartesianPositionToleranceMm</code>     | <code>double</code>   | 1.0        | Cartesian position tolerance in millimeters                                  |
| <code>CartesianOrientationToleranceDeg</code> | <code>double</code>   | 1.0        | Cartesian orientation tolerance in degrees                                   |

## Example

```
// Use default wait options (most common)
await robot.Move(MoveInstruction.MovJ(jp, speed: 40, acc: 100));

// Customize wait options for high-precision motion
var preciseWait = new MotionWaitOptions
{
    Timeout = TimeSpan.FromSeconds(120),
    PollInterval = TimeSpan.FromMilliseconds(20),
    SettledSamples = 5,
    JointToleranceDeg = 0.05,
    CartesianPositionToleranceMm = 0.2,
    CartesianOrientationToleranceDeg = 0.5,
};

await robot.Move(MoveInstruction.MovL(cp, speed: 50, acc: 200), preciseWait);

// Use a short timeout for quick motions
var quickWait = new MotionWaitOptions
{
    Timeout = TimeSpan.FromSeconds(10),
    PollInterval = TimeSpan.FromMilliseconds(30),
```

```
};

await robot.Move(MoveInstruction.MovJ(jp, speed: 100, acc: 300), quickWait);

// Fire-and-forget: set a very long timeout to effectively not wait
var longWait = new MotionWaitOptions
{
    Timeout = TimeSpan.FromHours(1),
};
```

## 9. RobotJogParameters

A sealed class defining parameters for robot jog (manual) movements.

`RobotJogParameters` specifies the jog mode (joint or linear), speed, axis index, and coordinate frame for manual robot jogging operations.

### Properties

| Property | Type              | Description                                  |
|----------|-------------------|----------------------------------------------|
| Mode     | RobotJogMode      | Jog mode: Joint or Linear                    |
| Speed    | double            | Jog speed (deg/s for joint, mm/s for linear) |
| Index    | int               | Axis index (0-5 for joint mode)              |
| CoorType | RobotJogFrameType | Coordinate frame type: User or Tool          |
| CoorId   | int               | Coordinate frame ID                          |

### Factory Method

| Method                                                                                                                  | Description           |
|-------------------------------------------------------------------------------------------------------------------------|-----------------------|
| <code>RobotJogParameters.Create(RobotJogMode mode, double speed, int index, RobotJogFrameType frame, int coorId)</code> | Create jog parameters |

### Example

```
// Jog joint 0 at 20 deg/s in user coordinate frame 0
var jogParams = RobotJogParameters.Create(
    mode: RobotJogMode.Joint,
    speed: 20,
    index: 0,
    frame: RobotJogFrameType.User,
    coorId: 0
);
await robot.Jog(jogParams);
```



```
// Jog linear along X axis at 50 mm/s in tool coordinate frame 1
var linearJog = RobotJogParameters.Create(
    mode: RobotJogMode.Linear,
    speed: 50,
    index: 0, // X axis
    frame: RobotJogFrameType.Tool,
    coordId: 1
);
await robot.Jog(linearJog);
```

## 10. RobotJogMode

An enum specifying the jog motion mode.

### Values

| Name   | Value | Description                     |
|--------|-------|---------------------------------|
| Joint  | 1     | Jog individual joints           |
| Linear | 2     | Jog linearly in Cartesian space |

### Example

```
// Switch between joint and linear jog modes
if (jogMode == RobotJogMode.Joint)
{
    Console.WriteLine("Joint jog mode - move individual joints.");
}
else if (jogMode == RobotJogMode.Linear)
{
    Console.WriteLine("Linear jog mode - move in Cartesian space.");
}
```

## 11. RobotJogFrameType

An enum specifying the coordinate frame type for jog operations.

### Values

| Name | Value | Description                   |
|------|-------|-------------------------------|
| User | 0     | User-defined coordinate frame |
| Tool | 1     | Tool coordinate frame         |

## Example

```
// Choose coordinate frame for jog operation
var jogParams = RobotJogParameters.Create(
    mode: RobotJogMode.Linear,
    speed: 30,
    index: 1, // Y axis
    frame: RobotJogFrameType.User, // Use user frame
    coordId: 0
);
await robot.Jog(jogParams);
```

## Complete Multi-Segment Path Example

The following example demonstrates building a complete motion program using multiple types from this API reference.

```
using CodroidCS.Sdk;

// 1. Define waypoints
var homeJoint = JointPoint.Degrees(new[] { 0, 0, 90, 0, 90, 0 });
var pickJoint = JointPoint.Degrees(new[] { 30, -20, 80, 10, 60, -5 });

var refJ = robot.CriData.JointPosition;
var pickCart = CartesianPoint.MmDegWithRef(
    new[] { 450, -100, 200, 180, 0, 0 },
    refJ
);
var placeCart = CartesianPoint.MmDeg(
    new[] { 450, 100, 200, 180, 0, 0 }
);
var viaPoint = CartesianPoint.MmDeg(
    new[] { 475, 0, 250, 180, 0, 0 }
);

// 2. Build a multi-segment path
var path = new[]
{
    // Move to home with joint motion
    MoveInstruction.MovJ(homeJoint, speed: 60, acc: 120),

    // Move to pick position with joint motion
    MoveInstruction.MovJ(pickJoint, speed: 40, acc: 100),

    // Linear approach to pick Cartesian point
    MoveInstruction.MovL(pickCart, speed: 80, acc: 300, blend: 5),

    // Arc motion from pick to place via waypoint
    MoveInstruction.MovC(viaPoint, placeCart, speed: 100, acc: 300),
}
```

```
// Return home with linear motion
MoveInstruction.MovL(
    CartesianPoint.MmDeg(new[] { 300, 0, 400, 180, 0, 0 }),
    speed: 150, acc: 500
),
};

// 3. Execute with custom wait options
var waitOptions = new MotionWaitOptions
{
    Timeout = TimeSpan.FromSeconds(90),
    SettledSamples = 4,
    JointToleranceDeg = 0.1,
};

await robot.Move(path, waitOptions);

// 4. Verify completion
Console.WriteLine("Path execution complete.");
```

# 5. Data Types and Enums

This section provides a comprehensive reference for all data types, enums, and exceptions in the Codroid CRI SDK.

## Table of Contents

1. [CommonResponse](#)
2. [CriRealTimeData](#)
3. [RobotFrame](#)
4. [RobotPayloadFrame](#)
5. [RobotParameters](#)
6. [RegisterReadValue](#)
7. [RegisterExtendArrayValueType](#)
8. [IoPortKind](#)
9. [RelativePoseCoorType](#)
10. [CodroidCommandException](#)
11. [GlobalVarSaveItem](#)
12. [GlobalVarRawJson](#)
13. [GlobalVarCatalogEntry](#)

## 1. CommonResponse

A general-purpose response class returned by most CRI SDK methods. Contains the request ID, a type identifier, a JSON data payload, and an optional error message.

### Properties

| Property | Type        | Description                  |
|----------|-------------|------------------------------|
| id       | object?     | The request identifier       |
| ty       | string?     | The response type identifier |
| db       | JsonElement | The response data payload    |
| err      | string?     | Error message, if any        |

## Example: Reading the db field

```
CommonResponse response = await client.SomeMethodAsync();

// Check for errors first
if (response.err != null)
{
    Console.WriteLine($"Error: {response.err}");
    return;
}

// Read db as a specific type
int value = response.db.GetInt32();
Console.WriteLine($"Value: {value}");

// Or read as JSON string
string json = response.db.GetRawText();
Console.WriteLine($"Raw JSON: {json}");
```

## 2. CriRealTimeData

Contains all real-time data fields from the robot controller, including joint positions, TCP poses, status flags, and more. Updated continuously via the CRI connection.

### Properties

#### Timestamp

| Property    | Type | Description               |
|-------------|------|---------------------------|
| TimestampMs | long | Timestamp in milliseconds |

#### Status Flags

| Property       | Type   | Description                            |
|----------------|--------|----------------------------------------|
| Status1Raw     | ushort | Raw status word 1                      |
| Status2Raw     | ushort | Raw status word 2                      |
| ProjectRunning | bool   | Whether a project is currently running |
| ProjectStopped | bool   | Whether the project is stopped         |
| ProjectPaused  | bool   | Whether the project is paused          |
| Enabling       | bool   | Whether the enabling switch is active  |
| NotEnabled     | bool   | Whether the robot is not enabled       |

| Property             | Type | Description                        |
|----------------------|------|------------------------------------|
| ManualMode           | bool | Whether in manual mode             |
| Dragging             | bool | Whether the robot is being dragged |
| InMotion             | bool | Whether the robot is in motion     |
| CollisionStopped     | bool | Whether stopped due to collision   |
| InSafetyPosition     | bool | Whether in the safety position     |
| HasAlarm             | bool | Whether an alarm is active         |
| SimulationMode       | bool | Whether in simulation mode         |
| EmergencyStopPressed | bool | Whether the E-stop is pressed      |
| RescueMode           | bool | Whether in rescue mode             |
| AutoMode             | bool | Whether in auto mode               |
| RemoteMode           | bool | Whether in remote mode             |
| RealTimeControlMode  | bool | Whether in real-time control mode  |
| CriErrorCode         | byte | CRI error code                     |

## Joint Data

| Property           | Type      | Description                |
|--------------------|-----------|----------------------------|
| JointPosition      | double[6] | Joint positions in degrees |
| JointVelocity      | double[6] | Joint velocities           |
| JointOutputTorque  | double[6] | Joint output torques       |
| JointExternalForce | double[6] | Joint external forces      |

## TCP Data

| Property          | Type      | Description                   |
|-------------------|-----------|-------------------------------|
| TcpPose           | double[6] | TCP pose (mm + deg)           |
| TcpVelocity       | double[6] | TCP velocity                  |
| TcpLinearVelocity | double    | TCP linear velocity magnitude |

## External

| Property                          | Type                  | Description             |
|-----------------------------------|-----------------------|-------------------------|
| <code>ExternalAxisPosition</code> | <code>double[]</code> | External axis positions |

## Methods

| Method                                   | Return Type                  | Description                             |
|------------------------------------------|------------------------------|-----------------------------------------|
| <code>UpdateFrom(CriRealTimeData)</code> | <code>void</code>            | Copies all fields from another instance |
| <code>Clone()</code>                     | <code>CriRealTimeData</code> | Creates a deep copy                     |

## Example: Subscribing to CRI data

```
// Subscribe to real-time data
client.OnCriDataReceived += (sender, data) =>
{
    // Read timestamp
    long timestamp = data.TimestampMs;

    // Read joint positions (in degrees)
    double joint1 = data.JointPosition[0];
    double joint2 = data.JointPosition[1];
    double joint3 = data.JointPosition[2];

    Console.WriteLine($"J1={joint1:F2}, J2={joint2:F2}, J3={joint3:F2}");

    // Read TCP pose
    double tcpX = data.TcpPose[0]; // mm
    double tcpY = data.TcpPose[1]; // mm
    double tcpZ = data.TcpPose[2]; // mm

    Console.WriteLine($"TCP X={tcpX:F2} Y={tcpY:F2} Z={tcpZ:F2}");

    // Check status
    if (data.HasAlarm)
    {
        Console.WriteLine("Robot has an alarm!");
    }
};

// Or clone for thread-safe access
CriRealTimeData snapshot = data.Clone();
```

### 3. RobotFrame

A sealed class representing a coordinate frame definition, used for both tool frames and user coordinate frames. Contains an ID and a 6-axis pose (position + orientation).

#### Properties

| Property        | Type                | Description                  |
|-----------------|---------------------|------------------------------|
| <code>Id</code> | <code>int</code>    | Frame identifier             |
| <code>X</code>  | <code>double</code> | X position (mm)              |
| <code>Y</code>  | <code>double</code> | Y position (mm)              |
| <code>Z</code>  | <code>double</code> | Z position (mm)              |
| <code>A</code>  | <code>double</code> | Rotation around X axis (deg) |
| <code>B</code>  | <code>double</code> | Rotation around Y axis (deg) |
| <code>C</code>  | <code>double</code> | Rotation around Z axis (deg) |

#### Example

```
// Access a tool frame
RobotFrame tool = robotParams.Tool[0];
Console.WriteLine($"Tool {tool.Id}: X={tool.X}, Y={tool.Y}, Z={tool.Z}");
Console.WriteLine($"  A={tool.A}, B={tool.B}, C={tool.C}");
```

### 4. RobotPayloadFrame

A sealed class representing a payload definition, including mass and center of mass coordinates.

#### Properties

| Property        | Type                | Description           |
|-----------------|---------------------|-----------------------|
| <code>Id</code> | <code>int</code>    | Payload identifier    |
| <code>M</code>  | <code>double</code> | Mass (kg)             |
| <code>Mx</code> | <code>double</code> | Center of mass X (mm) |
| <code>My</code> | <code>double</code> | Center of mass Y (mm) |
| <code>Mz</code> | <code>double</code> | Center of mass Z (mm) |



# Example

```
// Access a payload configuration
RobotPayloadFrame payload = robotParams.Payload[0];
Console.WriteLine($"Payload {payload.Id}: Mass={payload.M}kg");
Console.WriteLine($"  CoM: ({payload.Mx}, {payload.My}, {payload.Mz})");
```

## 5. RobotParameters

A sealed class containing the complete set of robot parameters, including default IDs for tool, payload, and coordinate frames, as well as the full lists of configured frames.

### Properties

| Property            | Type                    | Description                 |
|---------------------|-------------------------|-----------------------------|
| DefaultToolId       | int                     | Default tool frame ID       |
| DefaultPayloadId    | int                     | Default payload ID          |
| DefaultCoordinateId | int                     | Default coordinate frame ID |
| MaxPayload          | double                  | Maximum payload (kg)        |
| Tool                | List<RobotFrame>        | Tool frames list            |
| Payload             | List<RobotPayloadFrame> | Payload configurations list |
| Coordinate          | List<RobotFrame>        | User coordinate frames list |

### Example: Reading robot parameters

```
RobotParameters parameters = await client.GetRobotParametersAsync();

// Read defaults
Console.WriteLine($"Default Tool ID: {parameters.DefaultToolId}");
Console.WriteLine($"Default Payload ID: {parameters.DefaultPayloadId}");
Console.WriteLine($"Default Coordinate ID: {parameters.DefaultCoordinateId}");
Console.WriteLine($"Max Payload: {parameters.MaxPayload} kg");

// Iterate tool frames
Console.WriteLine("Tool Frames:");
foreach (RobotFrame tool in parameters.Tool)
{
    Console.WriteLine($"  [{tool.Id}] X={tool.X}, Y={tool.Y}, Z={tool.Z}, "
        + $"A={tool.A}, B={tool.B}, C={tool.C}");
}

// Iterate payloads
Console.WriteLine("Payloads:");
```

```

foreach (RobotPayloadFrame payload in parameters.Payload)
{
    Console.WriteLine($" [{payload.Id}] M={payload.M}kg, "
        + $"CoM=({payload.Mx}, {payload.My}, {payload.Mz})");
}

// Iterate coordinate frames
Console.WriteLine("Coordinate Frames:");
foreach (RobotFrame coord in parameters.Coordinate)
{
    Console.WriteLine($" [{coord.Id}] X={coord.X}, Y={coord.Y}, Z={coord.Z}, "
        + $"A={coord.A}, B={coord.B}, C={coord.C}");
}

```

## 6. RegisterReadValue

A readonly struct representing a value read from a controller register, with helpers to convert the value to common types.

### Properties

| Property | Type        | Description       |
|----------|-------------|-------------------|
| Address  | int         | Register address  |
| Value    | JsonElement | Raw value as JSON |

### Methods

| Method               | Return Type | Description                      |
|----------------------|-------------|----------------------------------|
| GetInt32()           | int         | Converts value to Int32          |
| GetDouble()          | double      | Converts value to Double         |
| TryGetInt32(out int) | bool        | Safely tries to convert to Int32 |

### Example: Reading and converting register values

```

// Read registers
List<RegisterReadValue> values = await client.ReadRegistersAsync(address: 0, count: 5);

foreach (RegisterReadValue reg in values)
{
    Console.WriteLine($"Register [{reg.Address}] raw: {reg.Value}");

    // Direct conversion (throws on failure)
    int intVal = reg.GetInt32();
    Console.WriteLine($" As Int32: {intVal}");
}

```

```
// Safe conversion
if (reg.TryGetInt32(out int safeVal))
{
    Console.WriteLine($" Safe Int32: {safeVal}");
}
else
{
    Console.WriteLine(" Cannot convert to Int32");
}

// As double
double dblVal = reg.GetDouble();
Console.WriteLine($" As Double: {dblVal}");
}
```

## 7. RegisterExtendArrayValueType

A static class defining constants for the data types used in extended register arrays.

### Constants

| Constant | Value | Description             |
|----------|-------|-------------------------|
| Bool     | 0     | Boolean type            |
| UInt8    | 1     | Unsigned 8-bit integer  |
| Int8     | 2     | Signed 8-bit integer    |
| UInt16   | 3     | Unsigned 16-bit integer |
| Int16    | 4     | Signed 16-bit integer   |
| UInt32   | 5     | Unsigned 32-bit integer |
| Int32    | 6     | Signed 32-bit integer   |
| Float32  | 7     | 32-bit floating point   |

### Example

```
// Specify value type when writing extended registers
await client.WriteExtendRegisterAsync(
    address: 0,
    values: new[] { 3.14f },
    valueType: RegisterExtendArrayValueType.Float32
);
```

## 8. IoPortKind

A static class defining constants for the different kinds of I/O ports available on the controller.

### Constants

| Constant | Value | Description    |
|----------|-------|----------------|
| Di       | "DI"  | Digital Input  |
| Do       | "DO"  | Digital Output |
| Ai       | "AI"  | Analog Input   |
| Ao       | "AO"  | Analog Output  |

### Example

```
// Read a digital input
bool diValue = await client.ReadDigitalInputAsync(port: IoPortKind.Di, address: 0);

// Write a digital output
await client.WriteDigitalOutputAsync(port: IoPortKind.Do, address: 0, value: true);

// Read an analog input
double aiValue = await client.ReadAnalogInputAsync(port: IoPortKind.Ai, address: 0);
```

## 9. RelativePoseCoorType

An enum specifying the coordinate system in which a relative pose is expressed.

### Values

| Name | Value | Description                    |
|------|-------|--------------------------------|
| User | 0     | User (world) coordinate system |
| Tool | 1     | Tool coordinate system         |

## Example

```
// Move relative in tool coordinate system
await client.MoveRelativeAsync(
    pose: new[] { 0, 0, 10, 0, 0, 0 }, // Move 10mm in Z
    coorType: RelativePoseCoorType.Tool
);

// Move relative in user coordinate system
await client.MoveRelativeAsync(
    pose: new[] { 10, 0, 0, 0, 0, 0 }, // Move 10mm in X
    coorType: RelativePoseCoorType.User
);
```

## 10. CodroidCommandException

A sealed exception class thrown when a CRI command fails. Provides detailed context about the failure including the request ID, command type, controller error message, and the full response.

### Inheritance

```
System.Exception
+-- CodroidCommandException
```

### Properties

| Property        | Type            | Description                     |
|-----------------|-----------------|---------------------------------|
| RequestId       | int             | The ID of the failed request    |
| CommandType     | string          | The type of command that failed |
| ControllerError | string?         | Error from the controller       |
| Response        | CommonResponse? | The full response object        |

### Constructor

```
public CodroidCommandException(
    int requestId,
    string commandType,
    string? controllerError,
    CommonResponse? response
)
```

# Example: Catching and inspecting

```
try
{
    await client.MoveJointAsync(target: new[] { 0, 0, 90, 0, 90, 0 });
}
catch (CodroidCommandException ex)
{
    Console.WriteLine("Command failed!");
    Console.WriteLine($" Request ID: {ex.RequestId}");
    Console.WriteLine($" Command Type: {ex.CommandType}");
    Console.WriteLine($" Controller Error: {ex.ControllerError}");

    if (ex.Response != null)
    {
        Console.WriteLine($" Response error: {ex.Response.err}");
        Console.WriteLine($" Response data: {ex.Response.db.GetRawText()}");
    }

    // Re-throw or handle
    throw;
}
```

## 11. GlobalVarSaveItem

A readonly record struct used to specify a global variable to be saved or written to the controller, including its name, value, and an optional remark.

### Constructor

```
public GlobalVarSaveItem(string Name, object Value, string? Remark = null)
```

| Parameter | Type    | Required           | Description     |
|-----------|---------|--------------------|-----------------|
| Name      | string  | Yes                | Variable name   |
| Value     | object  | Yes                | Variable value  |
| Remark    | string? | No (default: null) | Optional remark |

## Example

```
// Create items to save
var items = new List<GlobalVarSaveItem>
{
    new GlobalVarSaveItem("Counter", 42, "Production count"),
    new GlobalVarSaveItem("Speed", 50.5, "Motion speed"),
    new GlobalVarSaveItem("Flag", true)
};

await client.SaveGlobalVarsAsync(items);
```

## 12. GlobalVarRawJson

A readonly record struct that wraps a raw JSON string literal for writing global variables with complex or custom JSON structures.

### Constructor

```
public GlobalVarRawJson(string Literal)
```

| Parameter | Type   | Required | Description     |
|-----------|--------|----------|-----------------|
| Literal   | string | Yes      | Raw JSON string |

## Example

```
// Write a complex variable using raw JSON
var rawJson = new GlobalVarRawJson(
    """
    {"positions": [1.0, 2.0, 3.0], "enabled": true}
    """
);

await client.WriteGlobalVarRawAsync("MyConfig", rawJson);
```

## 13. GlobalVarCatalogEntry

A sealed class representing a single entry in the global variable catalog, containing the variable's current value and optional remark.

### Properties

| Property | Type        | Default | Description                  |
|----------|-------------|---------|------------------------------|
| Value    | JsonElement | --      | The variable's current value |

| Property | Type   | Default | Description           |
|----------|--------|---------|-----------------------|
| Remark   | string | ""      | Remark or description |

## Example

```
// Read the global variable catalog
Dictionary<string, GlobalVarCatalogEntry> catalog =
    await client.GetGlobalVarCatalogAsync();

foreach (var (name, entry) in catalog)
{
    Console.WriteLine($"Variable: {name}");
    Console.WriteLine($"  Value: {entry.Value.GetRawText()}");
    Console.WriteLine($"  Remark: {entry.Remark}");
}
```



# 6. CRI Real-Time Data and Control API Reference

This section covers the CRI (Codroid Real-time Interface) APIs for real-time robot control, trajectory generation, and data parsing.

## Table of Contents

- 1. [CriRealtimeDispatcher](#)
- 2. [TrajectoryGenerator](#)
- 3. [TrajectoryRequest](#)
- 4. [TrajectoryPoint](#)
- 5. [TrajectorySpace](#)
- 6. [TrajectoryProfile](#)
- 7. [CriRealtimePacketParser](#)
- 8. [Complete CRI Control Flow Example](#)

## 1. CriRealtimeDispatcher

Sealed class, implements `IDisposable`

A UDP-based command dispatcher that sends real-time motion commands to the robot controller. Supports single-frame commands and full trajectory playback with configurable SI-unit conversion.

### Constants

| Constant                              | Type             | Value             | Description                         |
|---------------------------------------|------------------|-------------------|-------------------------------------|
| <code>DefaultControllerUdpPort</code> | <code>int</code> | <code>9030</code> | Default UDP port for CRI commands   |
| <code>CommandPacketLength</code>      | <code>int</code> | <code>64</code>   | Fixed length of each command packet |

### Constructor

```
CriRealtimeDispatcher(string controllerIp, int controllerUdpPort = 9030, bool convertToSi = true)
```

| Parameter                      | Type                | Default           | Description                        |
|--------------------------------|---------------------|-------------------|------------------------------------|
| <code>controllerIp</code>      | <code>string</code> | <i>(required)</i> | IP address of the robot controller |
| <code>controllerUdpPort</code> | <code>int</code>    | <code>9030</code> | UDP port for sending commands      |

| Parameter                | Type              | Default           | Description                                                                                           |
|--------------------------|-------------------|-------------------|-------------------------------------------------------------------------------------------------------|
| <code>convertToSi</code> | <code>bool</code> | <code>true</code> | If <code>true</code> , converts deg to rad and mm to m before sending. Matches CRI data stream units. |

## Methods

### SendCommand

```
Task SendCommand(
    IReadOnlyList<double> position6,
    TrajectorySpace space,
    CancellationToken ct = default
)
```

Sends a single-frame position command to the controller. The `position6` list must contain exactly 6 elements (joint angles or Cartesian pose, depending on `space`).

| Parameter              | Type                                     | Description                                                    |
|------------------------|------------------------------------------|----------------------------------------------------------------|
| <code>position6</code> | <code>IReadOnlyList&lt;double&gt;</code> | Target position with exactly 6 elements                        |
| <code>space</code>     | <code>TrajectorySpace</code>             | Coordinate space: <code>Joint</code> or <code>Cartesian</code> |
| <code>ct</code>        | <code>CancellationToken</code>           | Cancellation token                                             |

#### Example:

```
// Send a single joint position command (degrees)
var dispatcher = new CriRealtimeDispatcher("192.168.8.136");
await dispatcher.SendCommand(
    new double[] { 0, 0, 90, 0, 90, 0 },
    TrajectorySpace.Joint
);
```

### SendTrajectory

```
Task SendTrajectory(
    IEnumerable<TrajectoryPoint> trajectory,
    TrajectorySpace space,
    int periodMs,
    CancellationToken ct = default
)
```

Sends a complete trajectory to the controller at a fixed time interval. Each point is dispatched according to `periodMs` to maintain real-time timing.

| Parameter               | Type                                            | Description                                                    |
|-------------------------|-------------------------------------------------|----------------------------------------------------------------|
| <code>trajectory</code> | <code>IEnumerable&lt;TrajectoryPoint&gt;</code> | Sequence of trajectory points to send                          |
| <code>space</code>      | <code>TrajectorySpace</code>                    | Coordinate space: <code>Joint</code> or <code>Cartesian</code> |
| <code>periodMs</code>   | <code>int</code>                                | Interval between points in milliseconds                        |
| <code>ct</code>         | <code>CancellationToken</code>                  | Cancellation token                                             |

**Example:**

```
// Send trajectory at 4ms intervals (250Hz)
using var dispatcher = new CriRealtimeDispatcher("192.168.8.136");
await dispatcher.SendTrajectory(trajectory, TrajectorySpace.Joint, periodMs: 4);
```

**Dispose**

```
void Dispose()
```

Closes the underlying UDP socket and releases resources. Call this when you are done sending commands, or use a `using` statement to ensure automatic cleanup.

## 2. TrajectoryGenerator

**Static class**

Generates smooth trajectories between two positions using configurable motion profiles (cubic or trapezoidal). Returns an enumerable sequence of `TrajectoryPoint` objects.

### Methods

**Generate**

```
static IEnumerable<TrajectoryPoint> Generate(
    IReadOnlyList<double> start,
    IReadOnlyList<double> target,
    TrajectoryRequest request
)
```

Generates a trajectory from `start` to `target` according to the parameters in `request`.

| Parameter            | Type                                     | Description                                   |
|----------------------|------------------------------------------|-----------------------------------------------|
| <code>start</code>   | <code>IReadOnlyList&lt;double&gt;</code> | Starting position (joint angles or Cartesian) |
| <code>target</code>  | <code>IReadOnlyList&lt;double&gt;</code> | Target position (joint angles or Cartesian)   |
| <code>request</code> | <code>TrajectoryRequest</code>           | Trajectory generation parameters              |

**Returns:** `IEnumerable<TrajectoryPoint>` -- sequence of trajectory points

**Example:**

```
var start = new double[] { 0, 0, 90, 0, 90, 0 };
var target = new double[] { 10, 20, 45, 0, 60, 30 };

var request = new TrajectoryRequest
{
    Space = TrajectorySpace.Joint,
    Profile = TrajectoryProfile.Cubic,
    FrequencyHz = 250,
    Speed = 30
};

var trajectory = TrajectoryGenerator.Generate(start, target, request).ToList();
Console.WriteLine($"Generated {trajectory.Count} trajectory points");
```

### 3. TrajectoryRequest

**Sealed class**

Parameters that control how a trajectory is generated, including coordinate space, timing, speed, and motion profile.

#### Properties

| Property        | Type              | Default | Description                                               |
|-----------------|-------------------|---------|-----------------------------------------------------------|
| Space           | TrajectorySpace   | (none)  | Coordinate space for the trajectory                       |
| FrequencyHz     | double            | 250.0   | Sampling frequency in Hz                                  |
| Speed           | double?           | null    | Target speed (mutually exclusive with DurationSeconds)    |
| DurationSeconds | double?           | null    | Total duration in seconds (mutually exclusive with Speed) |
| Profile         | TrajectoryProfile | Cubic   | Motion profile type                                       |
| Acceleration    | double            | 1000.0  | Acceleration value for trapezoidal profile                |

**Note:** `Speed` and `DurationSeconds` are mutually exclusive. Set only one of them. If both are set, behavior is undefined.

**Example:**

```
// Using Speed
var requestBySpeed = new TrajectoryRequest
{
```

```

    Space = TrajectorySpace.Joint,
    FrequencyHz = 250,
    Speed = 30,
    Profile = TrajectoryProfile.Trapezoidal,
    Acceleration = 800
};

// Using DurationSeconds
var requestByDuration = new TrajectoryRequest
{
    Space = TrajectorySpace.Joint,
    FrequencyHz = 250,
    DurationSeconds = 2.5,
    Profile = TrajectoryProfile.Cubic
};

```

## 4. TrajectoryPoint

### Sealed class

Represents a single point in a trajectory, containing a timestamp and a position array.

### Properties

| Property    | Type     | Default    | Description                                             |
|-------------|----------|------------|---------------------------------------------------------|
| TimeSeconds | double   | 0.0        | Time offset from trajectory start in seconds            |
| Position    | double[] | [] (empty) | Position values (joint angles or Cartesian coordinates) |

### Example:

```

var point = new TrajectoryPoint
{
    TimeSeconds = 0.016,
    Position = new double[] { 0.5, 1.0, 45.0, 0.0, 30.0, 10.0 }
};

Console.WriteLine($"t={point.TimeSeconds}s, pos=[{string.Join(", ", point.Position)}]");

```

## 5. TrajectorySpace

### Enum

Defines the coordinate space used for trajectory positions.

| Name  | Value | Description                                          |
|-------|-------|------------------------------------------------------|
| Joint | 0     | Joint space: positions are joint angles (deg or rad) |

| Name      | Value | Description                                                    |
|-----------|-------|----------------------------------------------------------------|
| Cartesian | 1     | Cartesian space: positions are tool pose (X, Y, Z, Rx, Ry, Rz) |

#### Example:

```
// Trajectory in joint space
var jointRequest = new TrajectoryRequest { Space = TrajectorySpace.Joint };

// Trajectory in Cartesian space
var cartRequest = new TrajectoryRequest { Space = TrajectorySpace.Cartesian };
```

## 6. TrajectoryProfile

### Enum

Defines the motion profile shape used during trajectory generation.

| Name        | Value | Description                                                                                          |
|-------------|-------|------------------------------------------------------------------------------------------------------|
| Cubic       | 0     | Cubic polynomial profile: smooth acceleration and deceleration                                       |
| Trapezoidal | 1     | Trapezoidal velocity profile: constant acceleration phase, cruise phase, constant deceleration phase |

#### Example:

```
// Cubic profile for smooth motion
var smoothRequest = new TrajectoryRequest
{
    Profile = TrajectoryProfile.Cubic,
    Speed = 30
};

// Trapezoidal profile with explicit acceleration
var preciseRequest = new TrajectoryRequest
{
    Profile = TrajectoryProfile.Trapezoidal,
    Speed = 50,
    Acceleration = 1000
};
```

## 7. CriRealtimePacketParser

### Static class

Parses raw CRI data packets received from the controller into structured `CriRealTimeData` objects. Automatically converts SI units (m to mm, rad to deg) for consistent use in application code.

# Constants

| Constant             | Type | Value | Description                                   |
|----------------------|------|-------|-----------------------------------------------|
| PacketLength         | int  | 308   | Expected length of a CRI data packet in bytes |
| DefaultDecimalPlaces | int  | 3     | Default number of decimal places for rounding |

# Methods

## Parse

```
static CriRealTimeData Parse(byte[] packet)
```

Parses a raw CRI data packet into a `CriRealTimeData` object. Converts m to mm and rad to deg for all position and orientation values.

| Parameter | Type   | Description                             |
|-----------|--------|-----------------------------------------|
| packet    | byte[] | Raw CRI data packet (must be 308 bytes) |

**Returns:** `CriRealTimeData` -- parsed real-time data object

## Example:

```
byte[] rawPacket = ReceiveCriPacket(); // your packet source
var data = CriRealtimePacketParser.Parse(rawPacket);

Console.WriteLine($"Joint Positions: [{string.Join(", ", data.JointPosition)}]");
Console.WriteLine($"TCP Pose: [{string.Join(", ", data.TcpPose)}]");
```

# 8. Complete CRI Control Flow Example

This example demonstrates the full CRI control workflow: starting data reception, reading the current position, generating a trajectory, sending commands via the real-time dispatcher, and cleanly shutting down.

```
using Codroid.CRI;
using Codroid.Robot;

// Initialize robot connection
var robot = new CodroidRobot("192.168.8.150");

// =====
// Step 1: Start CRI data push
// =====
// Begin receiving real-time data from the controller on the specified port
await robot.StartCriDataPush("192.168.8.150", 18888);

// =====
```

```

// Step 2: Read current position
// =====
// Use the current joint position as the trajectory start point
double[] start = robot.CriData.JointPosition;
Console.WriteLine($"Current position: [{string.Join(", ", start)}]");

// Define target position (joint angles in degrees)
double[] target = new[] { 0, 0, 90, 0, 90, 0 };
Console.WriteLine($"Target position: [{string.Join(", ", target)}]");

// =====
// Step 3: Generate trajectory
// =====
// Configure trajectory parameters
var request = new TrajectoryRequest
{
    Space = TrajectorySpace.Joint,          // Joint space
    Profile = TrajectoryProfile.Cubic,      // Smooth cubic profile
    FrequencyHz = 250,                      // 250Hz sampling
    Speed = 30                             // 30 deg/s
};

// Generate the trajectory
var trajectory = TrajectoryGenerator.Generate(start, target, request).ToList();
Console.WriteLine($"Generated {trajectory.Count} points over
{trajectory.Last().TimeSeconds:F3}s");

// =====
// Step 4: Start CRI control
// =====
// Enable real-time control mode on the controller
// filterType: 1 - Position filter type
// durationMs: 4 - Control loop period in ms
// startBuffer: 5 - Initial buffer size
await robot.StartCriControl(filterType: 1, durationMs: 4, startBuffer: 5);

try
{
    // =====
    // Step 5: Send trajectory
    // =====
    // Create a dispatcher and send the trajectory at 4ms intervals
    using var dispatcher = new CriRealtimeDispatcher("192.168.8.136");
    await dispatcher.SendTrajectory(trajectory, TrajectorySpace.Joint, periodMs: 4);

    Console.WriteLine("Trajectory execution completed");
}
finally
{
    // =====
    // Step 6: Stop CRI control
    // =====
    // Always stop CRI control and data push in finally block

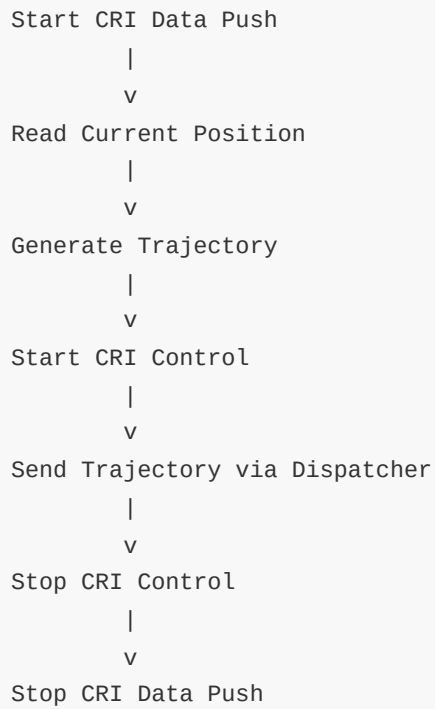
```



```
await robot.StopCriControl();
await robot.StopCriDataPush("192.168.8.150", 18888);

Console.WriteLine("CRI control stopped");
}
```

## Workflow Diagram



**Important:** Always stop CRI control and data push in a `finally` block to ensure clean shutdown, even if an exception occurs during trajectory execution.

# 7. IO and Register API Reference

---

## IO Operations

---

All IO methods are on `CodroidClient`.

---

### GetDi -- Read Digital Input

Read a digital input port. Returns `0` or `1`.

```
// Read DI port 0
int di0 = await robot.GetDi(0);
Console.WriteLine($"DI 0 = {di0}"); // 0 or 1
```

**Signature:**

```
Task<int> GetDi(int port)
```

---

### GetDo -- Read Digital Output

Read the current state of a digital output port. Returns `0` or `1`.

```
// Read DO port 10
int do10 = await robot.GetDo(10);
Console.WriteLine($"DO 10 = {do10}");
```

**Signature:**

```
Task<int> GetDo(int port)
```

---

### GetAi -- Read Analog Input

Read an analog input port. Returns a floating-point value.

```
// Read AI port 1
double ai1 = await robot.GetAi(1);
Console.WriteLine($"AI 1 = {ai1:F3}");
```

**Signature:**

```
Task<double> GetAi(int port)
```

---

## GetAo -- Read Analog Output

Read the current value of an analog output port.

```
// Read AO port 2
double ao2 = await robot.GetAo(2);
Console.WriteLine($"AO 2 = {ao2:F3}");
```

Signature:

```
Task<double> GetAo(int port)
```

---

## SetDo -- Write Digital Output

Write a digital output. `value` must be `0` or `1`.

```
// Set DO port 10 to ON
await robot.SetDo(10, 1);

// Set DO port 10 to OFF
await robot.SetDo(10, 0);
```

Signature:

```
Task<CommonResponse> SetDo(int port, int value)
```

Throws `ArgumentOutOfRangeException` if `value` is not `0` or `1`.

---

## SetAo -- Write Analog Output

Write an analog output value.

```
// Set AO port 2 to 3.14
await robot.SetAo(2, 3.14);
```

Signature:

```
Task<CommonResponse> SetAo(int port, double value)
```

---

## GetIoValues -- Batch Read

Batch-read multiple IO pins in a single request. Returns the raw `CommonResponse` whose `db` is a JSON array.

```
// Batch read DI0, DO10, AI1, AO2
var resp = await robot.GetIoValues(new List<(string Type, int Port)>
{
    (IoPortKind.Di, 0),
    (IoPortKind.Do, 10),
    (IoPortKind.Ai, 1),
    (IoPortKind.Ao, 2),
});
Console.WriteLine(resp.db.GetRawText());

// Parse individual values
int di0 = IoGetResponseParser.ParseDigital(resp, IoPortKind.Di, 0);
double ao2 = IoGetResponseParser.ParseAnalog(resp, IoPortKind.Ao, 2);
```

#### Signature:

```
Task<CommonResponse> GetIoValues(IReadOnlyList<(string Type, int Port)> pins)
```

## IO Helper Types

### IoPortKind

Constants for the `type` field in `IoManager` protocol.

| Constant                   | Value | Description    |
|----------------------------|-------|----------------|
| <code>IoPortKind.Di</code> | "DI"  | Digital input  |
| <code>IoPortKind.Do</code> | "DO"  | Digital output |
| <code>IoPortKind.Ai</code> | "AI"  | Analog input   |
| <code>IoPortKind.Ao</code> | "AO"  | Analog output  |

### IoGetResponseParser

Parses the `db` field from `GetIoValues` responses.

```
// Parse a digital value
int di = IoGetResponseParser.ParseDigital(response, IoPortKind.Di, 0);

// Parse an analog value
double ai = IoGetResponseParser.ParseAnalog(response, IoPortKind.Ai, 1);

// Build a query payload
var pins = new List<(string, int)> { (IoPortKind.Di, 0), (IoPortKind.Do, 10) };
JsonElement query = IoGetResponseParser.BuildGetQuery(pins);
```

| Method                                            | Returns                  | Description                                        |
|---------------------------------------------------|--------------------------|----------------------------------------------------|
| <code>ParseDigital(response, ioType, port)</code> | <code>int</code>         | Extract <code>0</code> or <code>1</code> for DI/DO |
| <code>ParseAnalog(response, ioType, port)</code>  | <code>double</code>      | Extract floating-point value for AI/AO             |
| <code>BuildGetQuery(pins)</code>                  | <code>JsonElement</code> | Build JSON array for batch IO query                |

## Register Operations

All register methods are on `CodroidClient`.

### GetRegisterValue -- Read Single Register

Read a single register. The returned `RegisterReadValue` contains the address and a raw JSON value. Use `GetInt32()` or `GetDouble()` to convert.

```
// Read register at address 49100
RegisterReadValue reg = await robot.GetRegisterValue(49100);

// Try as integer
if (reg.TryGetInt32(out int intVal))
    Console.WriteLine($"Address {reg.Address} = {intVal} (int)");
else
    Console.WriteLine($"Address {reg.Address} = {reg.GetDouble()} (float)");
```

Signature:

```
Task<RegisterReadValue> GetRegisterValue(int address)
```

### GetRegisterValues -- Batch Read

Read multiple registers in a single request. The returned list order matches the input addresses.

```
// Batch read registers 49100, 49102, 49104
var regs = await robot.GetRegisterValues(new[] { 49100, 49102, 49104 });

foreach (var r in regs)
{
    if (r.TryGetInt32(out int v))
        Console.WriteLine($" Address {r.Address}: {v}");
    else
        Console.WriteLine($" Address {r.Address}: {r.GetDouble():G}");
}
```

Signature:

```
Task<IReadOnlyList<RegisterReadValue>> GetRegisterValues(IReadOnlyList<int> addresses)
```

## SetRegisterValue (int) -- Write Integer

Write an integer value to a register.

```
// Write 520 to register 49100
await robot.SetRegisterValue(49100, 520);

// Write 0 to clear
await robot.SetRegisterValue(49100, 0);
```

**Signature:**

```
Task<CommonResponse> SetRegisterValue(int address, int value)
```

## SetRegisterValue (double) -- Write Float

Write a floating-point value to a register.

```
// Write 520.52 to register 49300
await robot.SetRegisterValue(49300, 520.52);

// Write 0.0 to clear
await robot.SetRegisterValue(49300, 0.0);
```

**Signature:**

```
Task<CommonResponse> SetRegisterValue(int address, double value)
```

## SetExtendArrayType -- Set Extended Array Element Type

Set the data type of an extend-array element. Index range: 0~999.

```
// Set element 0 to Int32
await robot.SetExtendArrayType(0, RegisterExtendArrayType.Int32);

// Set element 5 to Float32
await robot.SetExtendArrayType(5, RegisterExtendArrayType.Float32);
```

**Signature:**

```
Task<CommonResponse> SetExtendArrayType(int index, string type)
```

Throws `ArgumentOutOfRangeException` if `index` is not in 0~999.

Throws `ArgumentException` if `type` is not a recognized value.

## RemoveExtendArray -- Delete Extended Array Element

Remove an extend-array element and reset its data. Index range: 0~999.

```
// Remove element 0
await robot.RemoveExtendArray(0);
```

Signature:

```
Task<CommonResponse> RemoveExtendArray(int index)
```

## RegisterReadValue Struct

Holds the result of a register read: address and raw JSON value.

| Property | Type        | Description                        |
|----------|-------------|------------------------------------|
| Address  | int         | Register address                   |
| Value    | JsonElement | Raw JSON value from the controller |

| Method                     | Returns | Description                                |
|----------------------------|---------|--------------------------------------------|
| GetInt32()                 | int     | Read as integer; throws if not convertible |
| GetDouble()                | double  | Read as floating-point                     |
| TryGetInt32(out int value) | bool    | Try read as integer without throwing       |

```
RegisterReadValue reg = await robot.GetRegisterValue(49100);

// Safe integer read
if (reg.TryGetInt32(out int v))
    Console.WriteLine(v);

// Read as double
double d = reg.GetDouble();
```

## RegisterExtendArrayValueType Constants

Supported data types for extend-array elements.

| Constant                             | Wire Value |
|--------------------------------------|------------|
| RegisterExtendArrayValueType.Bool    | "Bool"     |
| RegisterExtendArrayValueType.UInt8   | "UInt8"    |
| RegisterExtendArrayValueType.Int8    | "Int8"     |
| RegisterExtendArrayValueType.UInt16  | "UInt16"   |
| RegisterExtendArrayValueType.Int16   | "Int16"    |
| RegisterExtendArrayValueType.UInt32  | "UInt32"   |
| RegisterExtendArrayValueType.Int32   | "Int32"    |
| RegisterExtendArrayValueType.Float32 | "Float32"  |

## Full Example: IO and Register Read-Write

```

using Codroid;

ConsoleUtf8.InitConsoleUtf8();

var robot = new CodroidClient("192.168.8.136");

try
{
    await robot.ConnectRemoteAndSwitchOn();

    // --- IO ---
    // Read DI and mirror to DO
    int di0 = await robot.GetDi(0);
    Console.WriteLine($"DI 0 = {di0}");
    await robot.SetDo(10, di0);
    Console.WriteLine($"DO 10 set to {di0}");

    // Read analog values
    double ai1 = await robot.GetAi(1);
    double ao2 = await robot.GetAo(2);
    Console.WriteLine($"AI 1 = {ai1:F3}, AO 2 = {ao2:F3}");

    // Batch IO
    var batch = await robot.GetIoValues(new List<(string, int)>
    {
        (IoPortKind.Di, 0),
        (IoPortKind.Do, 10),
    });
    Console.WriteLine("Batch result: " + batch.db.GetRawText());

    // --- Register ---
    // Single read

```



```
RegisterReadValue r = await robot.GetRegisterValue(49100);
Console.WriteLine($"Register 49100 = {r.GetDouble():G}");

// Batch read
var regs = await robot.GetRegisterValues(new[] { 49100, 49102, 49104 });
foreach (var rv in regs)
    Console.WriteLine($"    {rv.Address}: {rv.GetDouble():G}");

// Write integer
await robot.SetRegisterValue(49100, 520);

// Write float
await robot.SetRegisterValue(49300, 520.52);

// Extend array
await robot.SetExtendArrayType(0, RegisterExtendArrayType.Int32);
await robot.RemoveExtendArray(0);
}
finally
{
    robot.Disconnect();
}
```

## 8. Utilities API Reference

### Publish / Subscribe (TCP Topic Push)

The controller pushes state-change notifications over TCP. Use `SubscribePublishTopic` to register a callback for a specific topic.

#### SubscribePublishTopic

```
Task<PublishTopicSubscription> SubscribePublishTopic(  
    string topicTy,  
    Action<PublishNotification> handler,  
    int tcMilliseconds = 100)
```

Subscribes to a TCP publish topic. The first call on a connection sends a subscription frame (no `id`). Subsequent pushes matching `topicTy` are dispatched to `handler` on the thread pool.

| Parameter                   | Description                                                  |
|-----------------------------|--------------------------------------------------------------|
| <code>topicTy</code>        | Topic name, e.g. <code>PublishTopics.RobotStatus</code>      |
| <code>handler</code>        | Callback to process notifications; should not block for long |
| <code>tcMilliseconds</code> | Protocol <code>tc</code> field in ms; default 100            |

Returns a `PublishTopicSubscription` (disposable). Call `Dispose()` to unregister the local callback. It does NOT send an "unsubscribe" frame to the controller.

#### Example: Subscribe to RobotStatus

```
using Codroid;  
  
var robot = new CodroidClient("192.168.8.136");  
await robot.Connect();  
  
// Subscribe and collect pushes for 10 seconds  
int count = 0;  
using var sub = await robot.SubscribePublishTopic(  
    PublishTopics.RobotStatus,  
    msg =>  
    {  
        int n = Interlocked.Increment(ref count);  
        Console.WriteLine($"[{n}] ty={msg.Ty}");  
        if (msg.Db.ValueKind != JsonValueKind.Undefined)  
            Console.WriteLine(" db: " + msg.Db.GetRawText());  
    });  
  
await Task.Delay(TimeSpan.FromSeconds(10));  
Console.WriteLine($"Received {Volatile.Read(ref count)} pushes.");
```

```
sub.Dispose(); // Stop receiving locally
robot.Disconnect();
```

## PublishTopicSubscription

Disposable handle returned by `SubscribePublishTopic`. Disposing removes the local handler; the TCP connection being dropped also invalidates all subscriptions.

| Member                 | Description                            |
|------------------------|----------------------------------------|
| <code>TopicTy</code>   | The topic name ( <code>string</code> ) |
| <code>Dispose()</code> | Unregister the local callback          |

## PublishNotification

The notification object passed to your handler.

| Property             | Type                     | Description                                                      |
|----------------------|--------------------------|------------------------------------------------------------------|
| <code>Ty</code>      | <code>string</code>      | Topic type, e.g. <code>"publish/RobotStatus"</code>              |
| <code>Db</code>      | <code>JsonElement</code> | Business payload; <code>JsonValueKind.Undefined</code> if absent |
| <code>RawJson</code> | <code>string</code>      | Full JSON text of this message                                   |

## PublishTopics Constants

Common topic names for subscription.

| Constant                                   | Value                                  | Description             |
|--------------------------------------------|----------------------------------------|-------------------------|
| <code>PublishTopics.ProjectState</code>    | <code>"publish/ProjectState"</code>    | Project run state       |
| <code>PublishTopics.VarUpdate</code>       | <code>"publish/VarUpdate"</code>       | Global variable changed |
| <code>PublishTopics.RobotStatus</code>     | <code>"publish/RobotStatus"</code>     | Robot status            |
| <code>PublishTopics.RobotPosture</code>    | <code>"publish/RobotPosture"</code>    | Robot posture           |
| <code>PublishTopics.RobotCoordinate</code> | <code>"publish/RobotCoordinate"</code> | Coordinate data         |
| <code>PublishTopics.Log</code>             | <code>"publish/Log"</code>             | Log messages            |
| <code>PublishTopics.Error</code>           | <code>"publish/Error"</code>           | Error notifications     |

## PublishSubscribeDefaults

| Constant                                             | Value            | Description                                     |
|------------------------------------------------------|------------------|-------------------------------------------------|
| <code>PublishSubscribeDefaults.TcMilliseconds</code> | <code>100</code> | Default <code>tc</code> for subscription frames |

## Global Variables

### GetGlobalVars -- Read All

Read all global variables from the controller (raw response).

```
CommonResponse resp = await robot.GetGlobalVars();
Console.WriteLine(resp.db.GetRawText());
```

Signature:

```
Task<CommonResponse> GetGlobalVars()
```

### GetGlobalVarsCatalog -- Read Catalog

Read all global variables and parse them into a dictionary keyed by name.

```
var catalog = await robot.GetGlobalVarsCatalog();

foreach (var kv in catalog)
{
    Console.WriteLine($"  Name: {kv.Key}");
    Console.WriteLine($"  Value: {kv.Value.Value.GetRawText()}");
    Console.WriteLine($"  Remark: {kv.Value.Remark}");
}
```

Signature:

```
Task<IReadOnlyDictionary<string, GlobalVarCatalogEntry>> GetGlobalVarsCatalog()
```

### SaveGlobalVar -- Save Single

Save (create or update) a single global variable.

```
// Save an integer variable
await robot.SaveGlobalVar("my_counter", 100, "test remark");

// Save a string
await robot.SaveGlobalVar("my_name", "hello_codroid");
```

```
// Save an array
await robot.SaveGlobalVar("my_arr", new[] { 1, 2, 3 });

// Save a dictionary
await robot.SaveGlobalVar("my_map", new Dictionary<string, int> { ["x"] = 10 });

// Save raw JSON literal
await robot.SaveGlobalVar("my_pose",
    new GlobalVarRawJson("{\"jp\":[1,2,3,4,5,6]}"));
```

#### Signature:

```
Task<CommonResponse> SaveGlobalVar(string name, object value, string? remark = null)
```

Variable names are validated by `GlobalVarNaming.Validate()`: must start with a letter or underscore, contain only `[A-Za-z0-9_]`, must not start with `_`, and must not collide with reserved Lua/controller identifiers.

## SaveGlobalVars -- Batch Save

Save multiple global variables in one request.

```
await robot.SaveGlobalVars(new[]
{
    new GlobalVarSaveItem("sdk_test_int", 100, "integer test"),
    new GlobalVarSaveItem("sdk_test_float", 90.4, "float test"),
    new GlobalVarSaveItem("sdk_test_str", "hello", "string test"),
    new GlobalVarSaveItem("sdk_test_arr", new[] { 1, 2, 3 }),
});
```

#### Signature:

```
Task<CommonResponse> SaveGlobalVars(IReadOnlyCollection<GlobalVarSaveItem> items)
```

## RemoveGlobalVars -- Delete

Delete global variables by name. Deleting a non-existent name does not error.

```
await robot.RemoveGlobalVars(new[] { "sdk_test_int", "sdk_test_float" });
```

#### Signature:

```
Task<CommonResponse> RemoveGlobalVars(IEnumerable<string> names)
```

# Global Variable Helper Types

## GlobalVarSaveItem

Record struct for saving a variable.

```
public readonly record struct GlobalVarSaveItem(
    string Name,
    object Value,
    string? Remark = null);
```

| Field  | Description                                       |
|--------|---------------------------------------------------|
| Name   | Variable name (validated)                         |
| Value  | Any JSON-serializable object, or GlobalVarRawJson |
| Remark | Optional remark; null/blank omits the nm field    |

## GlobalVarRawJson

Wraps a pre-formatted JSON literal to avoid double-serialization.

```
var raw = new GlobalVarRawJson("{\"jp\": [1,2,3,4,5,6]}");
await robot.SaveGlobalVar("my_pose", raw);
```

## GlobalVarCatalogEntry

Parsed entry from GetGlobalVarsCatalog.

| Property | Type        | Description                  |
|----------|-------------|------------------------------|
| Value    | JsonElement | The variable's value         |
| Remark   | string      | Remark string; empty if none |

## GlobalVarNaming

Validation utilities for global variable names.

```
// Throws ArgumentException on invalid name
GlobalVarNaming.Validate("my_var");      // OK
GlobalVarNaming.Validate("__bad");        // throws: double underscore
GlobalVarNaming.Validate("if");          // throws: reserved word

// List of reserved names
IReadOnlyCollection<string> reserved = GlobalVarNaming.ReservedNames;
```

| Member                             | Description                                           |
|------------------------------------|-------------------------------------------------------|
| <code>Validate(string name)</code> | Throws <code>ArgumentException</code> on invalid name |
| <code>ReservedNames</code>         | Read-only collection of reserved identifiers          |

## Kinematics

Forward and inverse kinematics, and relative pose calculation.

### AposToCpos -- Forward Kinematics

Convert joint angles to Cartesian pose (forward kinematics).

```
var jp = new[] { 0.0, 0.0, 90.0, 0.0, 90.0, 0.0 };
var coor = new[] { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
var tool = new[] { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };

// Returns parsed result directly
double[] pose = await robot.AposToCposPose(jp, coor, tool);
// pose = [x, y, z, rx, ry, rz] in mm + deg
Console.WriteLine($"TCP: [{string.Join(", ", pose)}]");

// Or get raw response
CommonResponse resp = await robot.AposToCpos(jp, coor, tool);
```

#### Signature:

```
Task<double[]> AposToCposPose(
    double[] jointDegrees,
    double[] userFrame,
    double[] toolFrame,
    double[]? externalAxisPositions = null)
```

All vectors must be exactly 6 elements. Units: degrees for joints, mm + deg for frames.

### CposToApos -- Inverse Kinematics

Convert Cartesian pose to joint angles (inverse kinematics).

```

var cp = new[] { 927.503, 214.5, 898.998, 179.999, 0.0, -90.0 };
var rj = new[] { 10.0, 20.0, 30.0, 40.0, 50.0, 60.0 }; // reference joints

try
{
    double[] joints = await robot.CposToAposJoints(cp, rj);
    Console.WriteLine($"Joints (deg): [{string.Join(", ", joints)}]");
}
catch (InvalidOperationException)
{
    Console.WriteLine("No solution found. Try different reference joints.");
}

```

### Signature:

```

Task<double[]> CposToAposJoints(
    double[] cartesianMmDeg,
    double[] referenceJointDegrees,
    double[]? externalAxisPositions = null)

```

`referenceJointDegrees` is used as the starting guess. If the controller returns an empty array, an `InvalidOperationException` is thrown. Adjust the reference joints and retry.

## CalculateRelativePose / CalculateRelativePoseResult -- Relative Pose Calculation

Calculate an offset pose relative to a tool or user coordinate frame.

```

var currentPose = new[] { 927.503, 214.5, 898.998, 179.999, 0.0, -90.0 };
var offset = new[] { 0.0, 0.0, -300.0, 0.0, 0.0, 0.0 };

// Calculate in user coordinate frame
double[] result = await robot.CalculateRelativePoseResult(
    currentPose,
    offset,
    RelativePoseCoorType.User);

Console.WriteLine($"Result: [{string.Join(", ", result)}]");

// Or use tool coordinate frame
double[] toolResult = await robot.CalculateRelativePoseResult(
    currentPose,
    offset,
    RelativePoseCoorType.Tool);

```

### Signature:



```
Task<double[]> CalculateRelativePoseResult(
    double[] tcpPoseWorld,
    double[] offset,
    RelativePoseCoorType coorType,
    double[]? tcpPoseInPosCoorFrame = null,
    double[]? userCoorFrame = null)
```

## RelativePoseCoorType

| Value                     | Integer | Description           |
|---------------------------|---------|-----------------------|
| RelativePoseCoorType.User | 0       | User coordinate frame |
| RelativePoseCoorType.Tool | 1       | Tool coordinate frame |

## ConsoleUtf8

Sets `Console.InputEncoding` and `Console.OutputEncoding` to UTF-8. On Windows this prevents Chinese character garbling. On Linux/macOS it is a no-op.

### Signature:

```
public static void InitConsoleUtf8()
```

### Example:

```
using Codroid;

// Call at program entry
ConsoleUtf8.InitConsoleUtf8();

var robot = new CodroidClient("192.168.8.136");
// ... rest of your program
```

## Full Example: Utilities

```
using System;
using System.Text.Json;
using System.Threading;
using Codroid;

ConsoleUtf8.InitConsoleUtf8();

var robot = new CodroidClient("192.168.8.136");

try
{
    await robot.ConnectRemoteAndSwitchOn();
}
```

```

// --- Publish/Subscribe ---
using var sub = await robot.SubscribePublishTopic(
    PublishTopics.RobotStatus,
    msg => Console.WriteLine($"Push: ty={msg.Ty}"));

// --- Global Variables ---
await robot.SaveGlobalVar("sdk_demo", 42, "demo variable");
var catalog = await robot.GetGlobalVarsCatalog();
if (catalog.TryGetValue("sdk_demo", out var entry))
    Console.WriteLine($"sdk_demo = {entry.Value.GetRawText()}");

await robot.RemoveGlobalVars(new[] { "sdk_demo" });

// --- Kinematics ---
var jp = new[] { 0.0, 0.0, 90.0, 0.0, 90.0, 0.0 };
var zero = new[] { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };

double[] tcp = await robot.AposToCposPose(jp, zero, zero);
Console.WriteLine($"FK result: [{string.Join(", ", tcp)}]");

try
{
    double[] joints = await robot.CposToAposJoints(tcp, jp);
    Console.WriteLine($"IK result: [{string.Join(", ", joints)}]");
}
catch (InvalidOperationException)
{
    Console.WriteLine("IK: no solution with given reference.");
}

double[] offsetResult = await robot.CalculateRelativePoseResult(
    tcp, new[] { 0, 0, -100, 0, 0, 0 }, RelativePoseCoorType.Tool);
Console.WriteLine($"Relative pose: [{string.Join(", ", offsetResult)}]");

sub.Dispose();
}
finally
{
    robot.Disconnect();
}

```

## 9. .NET Framework 4.6.2 Notes

This section covers the platform constraints, timing behavior, polyfill layer, and build considerations when using the Codroid SDK on .NET Framework 4.6.2.

### 1. Platform Constraint

.NET Framework 4.6.2 runs only on **Windows**. Linux and macOS are not supported.

```
<!-- Only valid on Windows -->
<TargetFramework>net462</TargetFramework>
```

### 2. Target Framework

The SDK uses multi-targeting. The `net462` target is built alongside `net6.0` and `net8.0`.

```
<!-- From CodroidCS.csproj -->
<TargetFrameworks>net462;net6.0;net8.0</TargetFrameworks>
```

Your test project should reference the SDK and target `net462`:

```
<PropertyGroup>
  <OutputType>Exe</OutputType>
  <TargetFramework>net462</TargetFramework>
  <LangVersion>10</LangVersion>
</PropertyGroup>

<ItemGroup>
  <ProjectReference Include="..\CodroidSDK\CodroidCS.csproj" />
</ItemGroup>
```

### 3. CRI 250Hz SLA

The default and supported CRI real-time control frequency is **250Hz** (`periodMs = 4`).

- `periodMs = 4` is the **default SLA** -- tested and supported out of the box.
- `periodMs != 4` (including 500Hz / 1000Hz) is **NOT** within the default SLA. Higher frequencies require **on-site validation** of jitter and controller behavior.

When `periodMs != 4`, the SDK logs a warning via `Trace.TraceWarning`:

```
[Codroidsdk][net462] CRI SendTrajectory periodMs=N is outside the default 250Hz SLA.
The default supported period is 4ms. Higher-frequency or custom-period control
requires on-site validation.
```

## 4. Timing Implementation

On .NET 6+, `SendTrajectory` uses `PeriodicTimer`. On net462, `PeriodicTimer` is unavailable. The SDK falls back to:

1. `Stopwatch` -- high-resolution elapsed time measurement.
2. `Thread.Sleep(1)` -- for remaining time > 1.5ms (yields CPU).
3. `Thread.SpinWait(50)` -- for remaining time <= 1.5ms (busy-wait for precision).

```
// Simplified net462 timing loop
long ticksPerPeriod = (long)(Stopwatch.Frequency * periodMs / 1000.0);
long nextTick = stopwatch.ElapsedTicks;

foreach (var point in trajectory)
{
    await SendCommand(point.Position, space, ct);
    nextTick += ticksPerPeriod;
    long remainingTicks = nextTick - stopwatch.ElapsedTicks;

    if (remainingTicks > 0)
    {
        double remainingMs = remainingTicks * 1000.0 / Stopwatch.Frequency;
        if (remainingMs > 1.5)
            Thread.Sleep(1);          // coarse wait
        else
            Thread.SpinWait(50);      // fine wait
    }
}
```

## 5. Jitter Statistics

After `SendTrajectory` completes on net462, the SDK outputs jitter statistics via `Trace.TraceInformation`.

```
[Codroidsdk][net462] CRI SendTrajectory statistics:
Duration: 4.12s
Frames sent: 1031
Average period: 4.005ms
Max period: 5.823ms
Overruns (>6ms): 0
Max consecutive overruns: 0
UDP exceptions: 0
```

| Metric         | Description               |
|----------------|---------------------------|
| Duration       | Total elapsed time        |
| Frames sent    | Number of UDP frames sent |
| Average period | Mean inter-frame interval |

| Metric                   | Description                            |
|--------------------------|----------------------------------------|
| Max period               | Worst-case inter-frame interval        |
| Overruns (>6ms)          | Frames exceeding 6ms interval          |
| Max consecutive overruns | Longest streak of consecutive overruns |
| UDP exceptions           | Socket errors during send              |

## 6. TraceWarning for periodMs != 4

If you call `SendTrajectory` with any `periodMs` value other than 4, the SDK emits a `Trace.TraceWarning` before starting. This is informational only -- execution proceeds normally.

## 7. Polyfill Layer

Five polyfill files provide missing .NET 6+ APIs when targeting net462. All are in `CodroidSDK/Compat/`.

### 7.1 ArgumentNullException.ThrowIfNull

**File:** `Polyfills.cs`

On net462, `ArgumentNullException.ThrowIfNull` does not exist. The SDK provides `Polyfills.ThrowIfNull` with `CallerArgumentExpression` for automatic parameter name capture.

```
// Internal usage
Polyfills.ThrowIfNull(argument); // auto-captures param name

// On net6+ this delegates to ArgumentNullException.ThrowIfNull
```

### 7.2 Math.Clamp

**File:** `MathPolyfills.cs`

`Math.Clamp` is not available in .NET Framework. The SDK provides `MathPolyfills.Clamp` with identical semantics.

```
// int overload
int clamped = MathPolyfills.Clamp(value, min, max);

// double overload
double clampedD = MathPolyfills.Clamp(value, 0.0, 1.0);
```

## 7.3 double.IsFinite

**File:** DoublePolyfills.cs

`double.IsFinite` is not available in .NET Framework. The SDK provides `DoublePolyfills.IsFinite`.

```
bool ok = DoublePolyfills.IsFinite(d); // true if not NaN and not Infinity
```

## 7.4 IsExternalInit

**File:** IsExternalInit.cs

The `init` accessor keyword requires `System.Runtime.CompilerServices.IsExternalInit`, which does not exist in net462. This polyfill adds the marker type.

```
// Enables C# init properties on net462
public string Name { get; init; } = "";
```

## 7.5 CallerArgumentExpressionAttribute

**File:** CallerArgumentExpressionAttribute.cs

The `[CallerArgumentExpression]` attribute is a C# 10 / .NET 6+ feature. This polyfill declares the attribute for net462, enabling `Polyfills.ThrowIfNull` to capture parameter names automatically.

---

## 8. NuGet Dependencies

When targeting net462, the SDK pulls in two additional NuGet packages:

| Package                       | Version | Purpose                                                           |
|-------------------------------|---------|-------------------------------------------------------------------|
| <code>System.Text.Json</code> | 8.0.5   | JSON serialization                                                |
| <code>System.Memory</code>    | 4.5.5   | <code>Span&lt;T&gt;</code> , <code>Memory&lt;T&gt;</code> support |

```
<!-- From CodroidCS.csproj -->
<ItemGroup Condition="'$(TargetFramework)' == 'net462'">
  <PackageReference Include="System.Text.Json" Version="8.0.5" />
  <PackageReference Include="System.Memory" Version="4.5.5" />
</ItemGroup>
```

---

## 9. UdpClient Differences

The `UdpClient` API differs between .NET Framework and .NET 6+. The SDK uses conditional compilation to handle this.

## SendAsync

On net462, `UdpClient.SendAsync` is not available with `ReadOnlyMemory<byte>`. The SDK uses the synchronous `Send` method instead.

```
#if NET462
    _udp.Send(buffer, length, target);           // synchronous
    await Task.CompletedTask;
#else
    await _udp.SendAsync(buffer.AsMemory(), target, ct); // async
#endif
```

## ReceiveAsync with CancellationToken

On net462, `ReceiveAsync(CancellationToken)` is not available. The SDK registers a cancellation callback that closes the socket, causing the blocking `ReceiveAsync()` to throw.

```
#if NET462
    using var reg = token.Register(() =>
    {
        try { _udpClient?.Close(); } catch { }
    });
    // Then: await _udpClient.ReceiveAsync() (no token)
#else
    await _udpClient.ReceiveAsync(token);
#endif
```

## WriteDoubleLittleEndian

On net462, `BinaryPrimitives.WriteDoubleLittleEndian(Span<byte>, double)` is not available. The SDK falls back to `BitConverter.GetBytes` with manual endianness handling.

```
#if NET462
    byte[] bytes = BitConverter.GetBytes(value);
    if (!BitConverter.IsLittleEndian) Array.Reverse(bytes);
    Array.Copy(bytes, 0, buffer, offset, 8);
#else
    BinaryPrimitives.WriteDoubleLittleEndian(buffer.AsSpan(offset, 8), value);
#endif
```

---

## 10. LangVersion = 10

Both the SDK and net462 test projects set `<LangVersion>10</LangVersion>`. This enables C# 10 features (file-scoped namespaces, `init`, global usings, etc.) even on net462.

```
<PropertyGroup>
  <TargetFramework>net462</TargetFramework>
  <LangVersion>10</LangVersion>
  <ImplicitUsings>enable</ImplicitUsings>
  <Nullable>enable</Nullable>
</PropertyGroup>
```

---

## 11. Running the net462 Test Projects

### Basic API Test

```
# Full suite (IO, register, kinematics, publish, global vars)
dotnet run --project CodroidTestNet462/CodroidTestNet462.csproj -- 192.168.8.10

# Single IO test
dotnet run --project CodroidTestNet462/CodroidTestNet462.csproj -- io 192.168.8.10

# Single register test
dotnet run --project CodroidTestNet462/CodroidTestNet462.csproj -- register 192.168.8.10
```

### CRI Real-Time Control Test

```
# All segments (joint + cartesian + path)
dotnet run --project CodroidCRITestNet462/CodroidCRITestNet462.csproj

# Joint only
dotnet run --project CodroidCRITestNet462/CodroidCRITestNet462.csproj -- joint

# Cartesian with custom speed
dotnet run --project CodroidCRITestNet462/CodroidCRITestNet462.csproj -- cart --speed 120 --
accel 600

# Path with specific IPs
dotnet run --project CodroidCRITestNet462/CodroidCRITestNet462.csproj -- path 192.168.8.10
192.168.8.150

# Duration mode (6 seconds)
dotnet run --project CodroidCRITestNet462/CodroidCRITestNet462.csproj -- cart --duration 6
```

---

## Summary of net462 vs net6.0+ Differences

| Aspect                   | net462                                                                            | net6.0+                                  |
|--------------------------|-----------------------------------------------------------------------------------|------------------------------------------|
| Platform                 | Windows only                                                                      | Windows, Linux, macOS                    |
| CRI timer                | <code>Stopwatch</code> + <code>Thread.Sleep(1)</code> + <code>SpinWait(50)</code> | <code>PeriodicTimer</code>               |
| UDP send                 | <code>UdpClient.Send</code> (sync)                                                | <code>UdpClient.SendAsync</code> (async) |
| UDP receive cancellation | <code>token.Register(Close)</code>                                                | <code>ReceiveAsync(token)</code>         |



| Aspect                                  | net462                                                                 | net6.0+                                               |
|-----------------------------------------|------------------------------------------------------------------------|-------------------------------------------------------|
| Double write                            | <code>BitConverter.GetBytes</code> + manual endian                     | <code>BinaryPrimitives.WriteDoubleLittleEndian</code> |
| <code>Math.Clamp</code>                 | <code>MathPolyfills.Clamp</code>                                       | <code>Math.Clamp</code>                               |
| <code>double.IsFinite</code>            | <code>DoublePolyfills.IsFinite</code>                                  | <code>double.IsFinite</code>                          |
| <code>init</code> accessor              | Polyfill ( <code>IsExternalInit.cs</code> )                            | Built-in                                              |
| <code>[CallerArgumentExpression]</code> | Polyfill                                                               | Built-in                                              |
| NuGet extras                            | <code>System.Text.Json 8.0.5</code> , <code>System.Memory 4.5.5</code> | None                                                  |